

TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ CÔNG NGHỆ TP. HỒ CHÍ MINH
(UMT)
KHOA CÔNG NGHỆ — DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CẨM NANG BỒI DƯỠNG Olympic Tin Học & ICPC

*A Companion Guidebook: 18 Algorithmic Modules, C++ Implementation,
& Differential Testing*

ThS. Nguyễn Quân Bá Hồng

Department of Artificial Intelligence & Data Science, School of Technology, UMT
hong.nguyenquanba@umt.edu.vn

Academic Leadership & Scientific Advisory:

PGS. TS. Trần Đan Thư

Trưởng Khoa Công nghệ, UMT

Sản Phẩm Dạng II: Cẩm Nang Hướng Dẫn Bồi Dưỡng (Báo Cáo Thực Hành)
Nhiệm vụ Khoa học và Công nghệ Cấp Trường — Mã số Đề cương 2026/UMT-SOT

Thành phố Hồ Chí Minh, Năm 2026

Lời Nói Đầu & Quy Trình Thực Hành

Tài liệu này là Cẩm nang Bồi dưỡng (Sản phẩm Dạng II) thuộc Đề tài Khoa học và Công nghệ cấp Trường UMT: *Nghiên cứu xây dựng cẩm nang bồi dưỡng Olympic Tin học sinh viên trong thời đại AI* (Mã số đề cương: 2026/UMT-SOT).

Mỗi bài toán trong cẩm nang được cấu trúc theo Quy trình 3 Giai đoạn:

Giai đoạn 1: Phân Tích Toán Học: Phân tích cấu trúc đại số và tính chất đơn điệu (Monoid, tính chất Monge, cấu trúc DAG), thiết lập bất biến thuật toán và chứng minh giới hạn độ phức tạp thời gian và không gian.

Giai đoạn 2: Cài Đặt C++: Cài đặt thuật toán bằng ngôn ngữ C++, chú trọng cấu trúc dữ liệu mảng phẳng, tối ưu hóa tính cục bộ của bộ nhớ đệm (Cache Locality) và kiểm soát bộ nhớ.

Giai đoạn 3: Kiểm Thử Đối Chiếu (Python): Xây dựng kịch bản kiểm thử đối chiếu (Differential Testing) và sinh dữ liệu thử nghiệm ngẫu nhiên nhằm đối sánh kết quả giữa thuật toán tối ưu và thuật toán vét cạn (brute-force) làm chuẩn đối chứng (oracle).

Cuối mỗi chuyên đề là hệ thống câu hỏi mở rộng và thảo luận lý thuyết.

Mục lục

Lời Nói Đầu & Quy Trình Thực Hành	i
Chương 0: Nền Tảng Khoa Học Máy Tính Lý Thuyết	1
0.1 Mô Hình RAM & Giới Hạn Tính Toán	1
0.2 Định Nghĩa Ký Hiệu Tiệm Cận (Bachmann–Landau)	1
0.3 Lý Thuyết Phức Tạp Tính Toán ($\mathcal{P}$ vs $\mathcal{NP}$)	1
1 Kỹ Thuật Hai Con Trỏ (Two Pointers Method)	3
1.1 Bài Toán Mẫu & Phân Tích Bất Biến	3
1.2 Cài Đặt C++	4
1.3 Kiểm Thử Đối Chiếu & Thử Nghiệm Ngẫu Nhiên (Python)	4
1.4 Câu Hỏi Mở Rộng	5
2 Cửa Sổ Trượt & Hàng Đợi Đơn Điện (Sliding Window)	7
2.1 Bài Toán Mẫu & Phân Tích Bất Biến	7
2.2 Cài Đặt C++	7
2.3 Bộ Kiểm Thử So Lệch & Fuzzing Python	8
2.4 Câu Hỏi Mở Rộng	9
3 Mảng Cộng Dồn & Mảng Hiệu Đa Chiều (Prefix & Difference)	10
3.1 Bài Toán Mẫu & Phân Tích Bất Biến	10
3.2 Cài Đặt C++	10
3.3 Bộ Kiểm Thử So Lệch & Fuzzing Python	11
3.4 Câu Hỏi Mở Rộng	12
4 Chặt Nhị Phân Kết Quả (Binary Search on Answer)	13
4.1 Bài Toán Mẫu & Phân Tích Bất Biến	13
4.2 Cài Đặt C++	13
4.3 Bộ Kiểm Thử So Lệch & Fuzzing Python	14
4.4 Câu Hỏi Mở Rộng	14
5 Tập Hợp Không Rời Nhau Có Rollback (Rollback DSU)	15
5.1 Bài Toán Mẫu & Phân Tích Bất Biến	15
5.2 Cài Đặt C++	15
5.3 Bộ Kiểm Thử So Lệch & Fuzzing Python	16

5.4	Câu Hỏi Mở Rộng	17
6	Đồ Thị: Đường Đi Ngắn Nhất & 0-1 BFS	18
6.1	Bài Toán Mẫu & Phân Tích Bất Biến	18
6.2	Cài Đặt C++	18
6.3	Bộ Kiểm Thử So Lệnh & Fuzzing Python	19
6.4	Câu Hỏi Mở Rộng	19
7	Phân Rã Cây: Euler Tour & Binary Lifting LCA	20
7.1	Bài Toán Mẫu & Phân Tích Bất Biến	20
7.2	Cài Đặt C++	20
7.3	Bộ Kiểm Thử So Lệnh & Fuzzing Python	21
7.4	Câu Hỏi Mở Rộng	21
8	Quy Hoạch Động Bitmask & SOS (Sum over Subsets)	22
8.1	Bài Toán Mẫu & Phân Tích Bất Biến	22
8.2	Cài Đặt C++	22
8.3	Bộ Kiểm Thử So Lệnh & Fuzzing Python	22
8.4	Câu Hỏi Mở Rộng	23
9	Nhân Ma Trận & Bán vành Min-Plus	24
9.1	Bài Toán Mẫu & Phân Tích Bất Biến	24
9.2	Cài Đặt C++	24
9.3	Bộ Kiểm Thử So Lệnh & Fuzzing Python	25
9.4	Câu Hỏi Mở Rộng	26
10	Cây Phân Đoạn & Cập Nhật Lười (Lazy Segment Tree)	27
10.1	Bài Toán Mẫu & Phân Tích Bất Biến	27
10.2	Cài Đặt C++	27
10.3	Bộ Kiểm Thử So Lệnh & Fuzzing Python	28
10.4	Câu Hỏi Mở Rộng	29
11	Tối Ưu Hóa Quy Hoạch Động Lồi (Monge & Convexity)	30
11.1	Bài Toán Mẫu & Phân Tích Bất Biến	30
11.2	Cài Đặt C++	30
11.3	Bộ Kiểm Thử So Lệnh & Fuzzing Python	31
11.4	Câu Hỏi Mở Rộng	32
12	Lý Thuyết Trò Chơi & Hình Học Tính Toán	33
12.1	Bài Toán Mẫu & Phân Tích Bất Biến	33
12.2	Cài Đặt C++	33
12.3	Fuzzing Python	34
12.4	Câu Hỏi Mở Rộng	35

13 Dãy Con Tăng Dài Nhất (LIS) & Định Lý Mirsky	36
13.1 Bài Toán Mẫu & Phân Tích Bất Biến	36
13.2 Cài Đặt C++	36
13.3 Bộ Kiểm Thử So Lệnh & Fuzzing Python	37
13.4 Câu Hỏi Mở Rộng	37
14 Mạng Các Dòng Chảy & Cặp Ghép Đồ Thị Hai Phía (Flow Networks)	39
14.1 Bài Toán Mẫu & Phân Tích Bất Biến	39
14.2 Cài Đặt C++	39
14.3 Bộ Kiểm Thử So Lệnh & Fuzzing Python	41
14.4 Câu Hỏi Mở Rộng	41
15 Mảng Hậu Tổ & Mảng Tiền Tổ Chung Dài Nhất (Suffix Arrays & LCP)	42
15.1 Bài Toán Mẫu & Phân Tích Bất Biến	42
15.2 Cài Đặt C++	42
15.3 Bộ Kiểm Thử So Lệnh & Fuzzing Python	44
15.4 Câu Hỏi Mở Rộng	44
16 Tổ Hợp Nâng Cao & Lý Thuyết Nhóm (Combinatorics & Group Theory)	45
16.1 Bài Toán Mẫu & Phân Tích Bất Biến	45
16.2 Cài Đặt C++	45
16.3 Bộ Kiểm Thử So Lệnh & Fuzzing Python	46
16.4 Câu Hỏi Mở Rộng	47
17 Quy Hoạch Động Trên Cây & Không Gian Trạng Thái (Tree DP & State Space Search)	48
17.1 Bài Toán Mẫu & Phân Tích Bất Biến	48
17.2 Cài Đặt C++	48
17.3 Bộ Kiểm Thử So Lệnh & Fuzzing Python	49
17.4 Câu Hỏi Mở Rộng	51
18 Lớp Bài Toán $\mathcal{NP}$-Khó & Phép Thu Gọn Đa Thức (NP-Hardness & Reductions)	52
18.1 Bài Toán Mẫu & Phương Pháp Tiếp Cận	52
18.2 Cài Đặt C++	52
18.3 Bộ Kiểm Thử So Lệnh & Fuzzing Python	53
18.4 Câu Hỏi Mở Rộng	54
Phụ lục A: Tối Ưu Hóa Thuật Toán (C++)	55
Tài Liệu Tham Khảo	59

Chương 0: Nền Tảng Khoa Học Máy Tính Lý Thuyết

§ 0.1. Mô Hình RAM & Giới Hạn Tính Toán

Mọi thuật toán trong cẩm nang này được phân tích trên **Mô hình Máy Truy cập Ngẫu nhiên (Random Access Machine - RAM)** [Knu97]. Trong mô hình này:

- Không gian địa chỉ bộ nhớ là vô hạn và thời gian truy xuất mỗi ô nhớ là hằng số $\Theta(1)$.
- Các phép toán số học và thao tác bit cơ bản trên các từ máy kích thước w bit (với $w = \Omega(\log n)$) tiêu tốn thời gian $\Theta(1)$.

Trong các kiến trúc vi xử lý thực tế [Str13], cấu trúc phân cấp bộ nhớ (Memory Hierarchy) tạo ra độ trễ truy xuất không đồng đều. Cấu trúc dữ liệu yêu cầu ánh xạ vào các mảng bộ nhớ phẳng (Flat Arrays) để tối ưu hóa tỷ lệ trúng bộ nhớ đệm (Cache Hit rate).

§ 0.2. Định Nghĩa Ký Hiệu Tiệm Cận (Bachmann–Landau)

Độ phức tạp thuật toán được xác định qua hệ thống ký hiệu Bachmann–Landau [Knu97] đối với các hàm không âm tiệm cận $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

- **Cận trên tiệm cận:** $\mathcal{O}(g(n)) = \{f(n) \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n)\}$.
- **Cận dưới tiệm cận:** $\Omega(g(n)) = \{f(n) \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n \geq n_0, 0 \leq c \cdot g(n) \leq f(n)\}$.
- **Cận tiệm cận chặt (chính xác):** $\Theta(g(n)) = \mathcal{O}(g(n)) \cap \Omega(g(n))$, tương đương với sự tồn tại của các hằng số $c_1, c_2 > 0$ và $n_0 \in \mathbb{N}$ sao cho $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ với mọi $n \geq n_0$.

§ 0.3. Lý Thuyết Phức Tạp Tính Toán ($\mathcal{P}$ vs $\mathcal{NP}$)

Các bài toán quyết định (Decision Problems) được phân loại theo tính khả quy và khả năng giải trong thời gian đa thức:

- $\mathcal{P}$: Lớp các bài toán quyết định giải được bằng máy Turing đơn định trong thời gian đa thức $\mathcal{O}(n^k)$ với hằng số $k \geq 0$.

- $\mathcal{NP}$: Lớp các bài toán quyết định giải được bằng máy Turing phi đơn định trong thời gian đa thức, hoặc tương đương, tồn tại chứng chỉ nghiệm (certificate) có kích thước đa thức kiểm chứng được bằng máy Turing đơn định trong thời gian đa thức.

Đối với các bài toán thuộc lớp $\mathcal{NP}$ -khó (NP-hard), không tồn tại thuật toán thời gian đa thức $\mathcal{O}(n^k)$ giải đúng trong trường hợp tổng quát nếu giả thuyết $\mathcal{P} \neq \mathcal{NP}$ đúng.

Module 1

Kỹ Thuật Hai Con Trỏ (Two Pointers Method)

§ 1.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 1.1 (Subarray Sum on Monotonic Sequence). Cho mảng số nguyên dương $A = [a_1, a_2, \dots, a_n]$ ($a_i \geq 1, \forall i \in [n]$) có kích thước $n \geq 1$ và giá trị đích $S \in \mathbb{Z}^+$. Hãy xác định cặp chỉ số $1 \leq l \leq r \leq n$ sao cho $\sum_{i=l}^r a_i = S$, hoặc trả về $\{-1, -1\}$ nếu không tồn tại đoạn con thỏa mãn.

Giai đoạn 1: Phân tích Toán học

Xét dãy tổng tiền tố $P[k] = \sum_{i=1}^k a_i$ với quy ước $P[0] = 0$. Do mọi phần tử $a_i \geq 1 > 0$, dãy $\{P[k]\}_{k=0}^n$ là dãy số tăng ngặt trên $\mathbb{Z}_{\geq 0}$. Tổng của đoạn con bất kỳ $[l, r]$ ($1 \leq l \leq r \leq n$) được biểu diễn qua:

$$\sum_{i=l}^r a_i = P[r] - P[l-1]. \quad (1.1)$$

Với mỗi vị trí con trỏ phải r cố định, biểu thức $P[r] - P[l-1]$ là hàm giảm ngặt theo chỉ số con trỏ trái l . Tính chất đơn điệu này dẫn đến hai nhận xét cấu trúc:

- Nếu tại bước hiện tại tổng $\sum_{i=l}^r a_i > S$, thì với mọi chỉ số $l' < l$, ta có $\sum_{i=l'}^r a_i > \sum_{i=l}^r a_i > S$. Do đó, không tồn tại bất kỳ nghiệm hợp lệ nào kết thúc tại r với cận trái $l' \leq l$. Thuật toán tịnh tiến con trỏ trái: $l \leftarrow l + 1$.
- Khi con trỏ phải tịnh tiến từ r sang $r + 1$, tổng đoạn con tăng thêm $a_{r+1} \geq 1$. Nếu đoạn con kết thúc tại r với cận trái l đã thỏa mãn $\sum_{i=l}^r a_i \geq S$, thì với mọi $l' < l$, đoạn con $[l', r+1]$ sẽ có tổng $\sum_{i=l'}^{r+1} a_i > \sum_{i=l}^{r+1} a_i > S$. Vì vậy, cận trái hợp lệ tiếp theo $l_{\text{mới}}$ (nếu tồn tại) luôn thỏa mãn $l_{\text{mới}} \geq l$.

Bất biến đơn điệu này chứng minh rằng hai con trỏ l và r chỉ dịch chuyển tịnh tiến theo một chiều duy nhất từ trái sang phải mà không yêu cầu quay lui (backtracking). Trong mã nguồn cài đặt (C++ và Python), mảng được đánh chỉ số từ 0 ($0 \leq l \leq r < n$), do đó kết quả được quy đổi về chỉ số toán học 1-based thông qua cặp $\{l+1, r+1\}$.

Phân tích tổng hợp (Aggregate Analysis).

Quá trình thực thi của thuật toán được phân tích thông qua chuyển động phối hợp giữa hai con trỏ l và r :

- Con trỏ phải r duyệt tịnh tiến đơn điệu từ chỉ số 0 đến $n - 1$, thực hiện tối đa n lần lặp và tăng chỉ số.
- Con trỏ trái l được khởi tạo tại 0 và chỉ tăng khi tổng hiện tại vượt quá S . Do điều kiện dừng $l \leq r$ và mỗi bước tăng l làm giảm tổng, con trỏ l không bao giờ vượt quá n . Tổng số lần thực hiện phép tăng $l++$ trong suốt toàn bộ thời gian chạy bị chặn trên bởi n .

Tổng số bước dịch chuyển của cả hai con trỏ trong trường hợp xấu nhất không vượt quá $2n$. Vì mỗi bước dịch chuyển chỉ thực hiện các phép toán số học và so sánh trong thời gian $\mathcal{O}(1)$, độ phức tạp thời gian trong trường hợp xấu nhất (worst-case time complexity) của thuật toán được giới hạn bởi $\mathcal{O}(n)$.

Mặt khác, trong trường hợp tốt nhất (khi ngay phần tử đầu tiên $a_1 = S$), thuật toán trả về kết quả ngay tại bước lặp đầu tiên với thời gian $\Omega(1)$. Do đó, thời gian thực thi tổng quát của thuật toán là $\mathcal{O}(n)$, và chi phí không gian bộ nhớ phụ trợ là $\mathcal{O}(1)$. $\square$

§ 1.2. Cài Đặt C++

```

1 #include <iostream>
2 #include <vector>
3 #include <utility>
4
5 using namespace std;
6
7 pair<int, int> find_subarray_sum(const vector<long long>& a, long long S) {
8     if (S <= 0) return {-1, -1};
9     int n = a.size();
10    int l = 0;
11    long long current_sum = 0;
12
13    for (int r = 0; r < n; ++r) {
14        current_sum += a[r];
15        while (current_sum > S && l <= r) {
16            current_sum -= a[l];
17            l++;
18        }
19        if (current_sum == S) {
20            return {l + 1, r + 1}; // 1-based indexing
21        }
22    }
23    return {-1, -1};
24 }
```

Listing 1.1: C++ Implementation of Two Pointers Subarray Sum

§ 1.3. Kiểm Thử Đối Chiếu & Thử Nghiệm Ngẫu Nhiên (Python)

```

1 import random
2
3 def brute_force_subarray_sum(a, S):
4     if S <= 0: return (-1, -1)
5     n = len(a)
6     for l in range(n):
7         s = 0
8         for r in range(l, n):
9             s += a[r]
10            if s == S:
11                return (l + 1, r + 1)
12    return (-1, -1)
13
14 def two_pointers_solve(a, S):
15     if S <= 0: return (-1, -1)
16     n = len(a)
17     l, cur = 0, 0
18     for r in range(n):
19         cur += a[r]
20         while cur > S and l <= r:
21             cur -= a[l]
22             l += 1
23         if cur == S:
24             return (l + 1, r + 1)
25    return (-1, -1)
26
27 def run_stress_test(num_trials=1000):
28     for trial in range(num_trials):
29         n = random.randint(1, 200)
30         a = [random.randint(1, 50) for _ in range(n)]
31         S = random.randint(1, sum(a) + 10)
32         ans_brute = brute_force_subarray_sum(a, S)
33         ans_opt = two_pointers_solve(a, S)
34         # Verify valid sum if indices found
35         if ans_opt != (-1, -1):
36             l, r = ans_opt
37             assert sum(a[l-1:r]) == S, f"invalid sum on trial {trial}"
38         else:
39             assert ans_brute == (-1, -1), f"Missed valid subarray on trial {
40 trial}"
41         print(f"Stress test passed {num_trials} randomized trials successfully.")
42
43 if __name__ == "__main__":
44     run_stress_test()

```

Listing 1.2: Python Differential Testbench for Two Pointers

§ 1.4. Câu Hỏi Mở Rộng

1. **Tính chất đơn điệu:** Khi mảng A chứa các phần tử mang dấu âm ($a_i < 0$), vì sao tính đơn điệu của tổng tiền tố $P[k]$ bị phá vỡ? Để giải quyết bài toán tìm đoạn con có tổng bằng S trên miền giá trị số nguyên tùy ý, cấu trúc dữ liệu bảng băm tiền tố (`std::unordered_map`) đạt độ phức tạp thời gian kỳ vọng là bao nhiêu?

2. **Ràng buộc biên:** Trong trường hợp mảng cho phép phần tử bằng 0 ($a_i \geq 0$) hoặc giá trị đích $S = 0$, việc thiếu điều kiện dừng $l \leq r$ trong vòng lặp **while** sẽ dẫn đến lỗi logic nào đối với thứ tự hai con trỏ?

Module 2

Cửa Sổ Trượt & Hàng Đợi Đơn Điều (Sliding Window)

§ 2.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 2.1 (Sliding Window Minimum). Cho mảng A có N phần tử và số nguyên $K \leq N$. Tìm giá trị nhỏ nhất trong mỗi cửa sổ trượt có độ dài K : $\min(a_{i-K+1}, \dots, a_i)$ với mỗi $K \leq i \leq N$.

Giai đoạn 1: Phân tích Toán học

Khi xét phần tử mới a_i đi vào cửa sổ, các phần tử a_j cũ hơn ($j < i$) có giá trị $a_j \geq a_i$ không thể trở thành cực tiểu của bất kỳ cửa sổ nào trong tương lai. Ta loại bỏ a_j khỏi hàng đợi. Đồng thời, khi các phần tử hết hạn (rơi khỏi cửa sổ kích thước K , tức là $j \leq i - K$), chúng sẽ bị loại bỏ khỏi đầu hàng đợi (evicted from the front). Hàng đợi duy trì các chỉ số luôn có giá trị tương ứng tăng ngặt, cho phép truy xuất phần tử nhỏ nhất ở đầu hàng đợi trong $\Theta(1)$. Mỗi phần tử trong mảng được đẩy vào deque đúng một lần. Do đó, số lần bị loại bỏ (pop) không thể vượt quá N , giới hạn tổng số thao tác cấu trúc dữ liệu nội tại, đạt độ phức tạp tiệm cận $\Theta(N)$.

§ 2.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <deque>
4 using namespace std;
5
6 vector<int> sliding_window_minimum(const vector<int>& a, int k) {
7     if (k <= 0) return {};
8     int n = a.size();
9     vector<int> result;
10    deque<int> dq; // Stores indices
11
12    for (int i = 0; i < n; ++i) {
13        // remove elements outside current window [i - k + 1, i]
14        if (!dq.empty() && dq.front() <= i - k) {
15            dq.pop_front();
16        }
17        // Maintain monotonic increasing order of values
```

```

18 while (!dq.empty() && a[dq.back()] >= a[i]) {
19     dq.pop_back();
20 }
21 dq.push_back(i);
22 // record answer once the first window of size k is formed
23 if (i >= k - 1) {
24     result.push_back(a[dq.front()]);
25 }
26 }
27 return result;
28 }

```

Listing 2.1: C++ Monotonic Deque for Sliding Window Minimum

§ 2.3. Bộ Kiểm Thử So Lệch & Fuzzing Python

```

1 import random
2 from collections import deque
3
4 def brute_force_sliding_min(a, k):
5     if k <= 0: return []
6     return [min(a[i:i+k]) for i in range(len(a) - k + 1)]
7
8 def deque_sliding_min(a, k):
9     if k <= 0: return []
10    dq = deque()
11    res = []
12    for i, x in enumerate(a):
13        if dq and dq[0] <= i - k:
14            dq.popleft()
15        while dq and a[dq[-1]] >= x:
16            dq.pop()
17        dq.append(i)
18        if i >= k - 1:
19            res.append(a[dq[0]])
20    return res
21
22 def stress_test(num_trials=500):
23     for trial in range(num_trials):
24         n = random.randint(1, 300)
25         k = random.randint(1, n)
26         if random.random() < 0.05:
27             k = random.randint(-5, 0)
28         a = [random.randint(-1000, 1000) for _ in range(n)]
29         expected = brute_force_sliding_min(a, k)
30         actual = deque_sliding_min(a, k)
31         assert expected == actual, f"Mismatch on trial {trial}"
32         print(f"Sliding window minimum passed {num_trials} stress tests.")
33
34 if __name__ == "__main__":
35     stress_test()

```

Listing 2.2: Python Differential Fuzzer for Sliding Window

§ 2.4. Câu Hỏi Mở Rộng

1. *Kiến trúc:* Làm thế nào để thay thế `std::deque` bằng hai con trỏ trên mảng tĩnh phẳng nhằm giảm đáng kể hằng số thời gian thực thi?
2. *Mở rộng:* Mở rộng bài toán tìm cực trị cửa sổ trượt 2D kích thước $K \times M$ trên ma trận $N \times P$ trong $\Theta(N \cdot P)$.

Module 3

Mảng Cộng Dồn & Mảng Hiệu Đa Chiều (Prefix & Difference)

§ 3.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 3.1 (2D Range Update and Range Query). Cho lưới $N \times M$ ban đầu toàn số 0. Thực hiện Q_1 thao tác cộng giá trị V vào hình chữ nhật $[x_1, y_1] \times [x_2, y_2]$, sau đó thực hiện Q_2 truy vấn tính tổng các phần tử trong hình chữ nhật $[r_1, c_1] \times [r_2, c_2]$.

Giai đoạn 1: Phân tích Toán học

Mảng hiệu D đóng vai trò là đạo hàm rời rạc, còn mảng cộng dồn $Pref$ là tích phân rời rạc trên lưới $\mathbb{Z}^2$. Dựa trên nguyên lý bù trừ 2D (2D Inclusion-Exclusion Principle), việc cộng V vào vùng $[x_1, x_2] \times [y_1, y_2]$ tương đương với việc điều chỉnh tại 4 điểm nằm ngay ngoài biên hình chữ nhật:

$$D[x_1][y_1] \leftarrow D[x_1][y_1] + V, \quad D[x_2 + 1][y_1] \leftarrow D[x_2 + 1][y_1] - V \quad (3.1)$$

$$D[x_1][y_2 + 1] \leftarrow D[x_1][y_2 + 1] - V, \quad D[x_2 + 1][y_2 + 1] \leftarrow D[x_2 + 1][y_2 + 1] + V \quad (3.2)$$

Thuật toán đạt độ phức tạp thời gian tiệm cận $\mathcal{O}(1)$ cho thao tác cập nhật, $\mathcal{O}(N \times M)$ cho bước dựng mảng, và $\mathcal{O}(1)$ cho truy vấn. Với giới hạn phổ biến $N, M \leq 1000$, ta chọn hằng số mảng $MAX=1005$. Chú ý cần cấp phát mảng với kích thước $+2$ (hoặc lẻ bảo vệ) để tránh lỗi tràn chỉ số (*out-of-bounds*) khi truy cập $(x_2 + 1, y_2 + 1)$.

§ 3.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 const int MAX = 1005;
6 long long D[MAX][MAX], Pref[MAX][MAX];
7
8 void range_add_2d(int x1, int y1, int x2, int y2, long long v) {
9     D[x1][y1] += v;
10    D[x2 + 1][y1] -= v;
```

```

11     D[x1][y2 + 1] -= v;
12     D[x2 + 1][y2 + 1] += v;
13 }
14
15 void build_2d(int n, int m) {
16     for (int i = 1; i <= n; ++i) {
17         for (int j = 1; j <= m; ++j) {
18             D[i][j] += D[i - 1][j] + D[i][j - 1] - D[i - 1][j - 1];
19             Pref[i][j] = D[i][j] + Pref[i - 1][j] + Pref[i][j - 1] - Pref[i -
20             1][j - 1];
21         }
22     }
23 }
24 long long query_sum_2d(int x1, int y1, int x2, int y2) {
25     return Pref[x2][y2] - Pref[x1 - 1][y2] - Pref[x2][y1 - 1] + Pref[x1 - 1][y1
26     - 1];
27 }

```

Listing 3.1: C++ Implementation of 2D Difference Arrays and Prefix Sums

§ 3.3. Bộ Kiểm Thử So Lệch & Fuzzing Python

```

1  import random
2
3  def solve_grid_naive(n, m, updates, queries):
4      grid = [[0] * (m + 1) for _ in range(n + 1)]
5      for x1, y1, x2, y2, v in updates:
6          for i in range(x1, x2 + 1):
7              for j in range(y1, y2 + 1):
8                  grid[i][j] += v
9
10     res = []
11     for x1, y1, x2, y2 in queries:
12         s = sum(grid[i][j] for i in range(x1, x2 + 1) for j in range(y1, y2 + 1)
13         )
14         res.append(s)
15     return res
16
17 def solve_grid_prefix(n, m, updates, queries):
18     D = [[0] * (m + 2) for _ in range(n + 2)]
19     for x1, y1, x2, y2, v in updates:
20         D[x1][y1] += v
21         D[x2+1][y1] -= v
22         D[x1][y2+1] -= v
23         D[x2+1][y2+1] += v
24
25     Pref = [[0] * (m + 1) for _ in range(n + 1)]
26     for i in range(1, n + 1):
27         for j in range(1, m + 1):
28             D[i][j] += D[i-1][j] + D[i][j-1] - D[i-1][j-1]
29             Pref[i][j] = D[i][j] + Pref[i-1][j] + Pref[i][j-1] - Pref[i-1][j-1]
30
31     res = []
32     for x1, y1, x2, y2 in queries:

```

```

31     s = Pref[x2][y2] - Pref[x1-1][y2] - Pref[x2][y1-1] + Pref[x1-1][y1-1]
32     res.append(s)
33     return res
34
35 if __name__ == "__main__":
36     n, m = 20, 20
37     updates = []
38     for _ in range(30):
39         x_a, x_b = random.randint(1, n), random.randint(1, n)
40         y_a, y_b = random.randint(1, m), random.randint(1, m)
41         updates.append((min(x_a, x_b), min(y_a, y_b), max(x_a, x_b), max(y_a,
42 y_b), random.randint(1, 100)))
43     queries = []
44     for _ in range(50):
45         x_a, x_b = random.randint(1, n), random.randint(1, n)
46         y_a, y_b = random.randint(1, m), random.randint(1, m)
47         queries.append((min(x_a, x_b), min(y_a, y_b), max(x_a, x_b), max(y_a,
48 y_b)))
49     assert solve_grid_naive(n, m, updates, queries) == solve_grid_prefix(n, m,
updates, queries)
50     print("2D Prefix and Difference testing passed with exact precision.")

```

Listing 3.2: Python 2D Differential Grid Validator

§ 3.4. Câu Hỏi Mở Rộng

1. *Bất biến*: Công thức mảng hiệu 3D gồm bao nhiêu phép gán góc? (Trả lời: $2^3 = 8$ góc theo nguyên lý bù trừ).
2. *Tối ưu Cache*: Tại sao cần duyệt chỉ số i trước j trong bộ nhớ C++?

Module 4

Chặt Nhị Phân Kết Quả (Binary Search on Answer)

§ 4.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 4.1 (Factory Machines Scheduling). Có N máy móc, máy thứ i mất $k_i \geq 1$ giây để làm ra 1 sản phẩm. Hãy tìm thời gian tối thiểu T để cả N máy cùng sản xuất đủ P sản phẩm.

Giai đoạn 1: Phân tích Toán học

Gọi $f(t) = \sum_{i=1}^N \lfloor \frac{t}{k_i} \rfloor$ là số sản phẩm làm được sau thời gian t . Vì $k_i \geq 1$, $f(t)$ là hàm đơn điệu không giảm (monotonically non-decreasing) theo t . Điều kiện $f(t) \geq P$ phân chia không gian tìm kiếm thành hai nửa: tồn tại ngưỡng T sao cho $f(t) < P, \forall t < T$ và $f(t) \geq P, \forall t \geq T$.

§ 4.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 bool check(long long t, const vector<long long>& k, long long P) {
7     long long total = 0;
8     for (long long time_req : k) {
9         total += t / time_req;
10        if (total >= P) return true; // Avoid integer overflow
11    }
12    return total >= P;
13 }
14
15 long long min_time_for_products(const vector<long long>& k, long long P) {
16     if (k.empty()) return P == 0 ? 0 : -1;
17     long long max_k = *max_element(k.begin(), k.end());
18     long long low = 0, high = P * max_k;
19     long long ans = high;
20
21     while (low <= high) {
```

```

22 long long mid = low + (high - low) / 2;
23 if (check(mid, k, P)) {
24     ans = mid;
25     high = mid - 1; // Try smaller time
26 } else {
27     low = mid + 1; // Increase time
28 }
29 }
30 return ans;
31 }

```

Listing 4.1: C++ Binary Search on Monotonic Predicate

§ 4.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2
3 def check_valid(t, k, P):
4     return sum(t // x for x in k) >= P
5
6 def solve_bs(k, P):
7     if not k:
8         return 0 if P == 0 else -1
9     low, high = 0, P * max(k)
10    ans = high
11    while low <= high:
12        mid = (low + high) // 2
13        if check_valid(mid, k, P):
14            ans = mid
15            high = mid - 1
16        else:
17            low = mid + 1
18    return ans
19
20 def test_extreme_cases():
21     k = [1] * 100000
22     P = 10**9
23     ans = solve_bs(k, P)
24     assert ans == 10000
25     print("Extreme boundary schedule test passed.")
26
27 if __name__ == "__main__":
28     test_extreme_cases()

```

Listing 4.2: Python Fuzzer for Monotonic Predicate Invariant

§ 4.4. Câu Hỏi Mở Rộng

1. Phép tính $(low + high) / 2$ gây lỗi tràn số (overflow) trong C++ như thế nào khi biến $high = 2 \times 10^9$?

Module 5

Tập Hợp Không Rời Nhau Có Rollback (Rollback DSU)

§ 5.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 5.1 (Incremental Connectivity với Rollback). Cho đồ thị N đỉnh. Hỗ trợ 3 loại thao tác: thêm cạnh (u, v) , truy vấn hai đỉnh u, v có liên thông không, và rút lui (undo) thao tác thêm cạnh vừa thực hiện.

Phương pháp nén đường đi (*Path Compression*) làm phá vỡ cấu trúc hình học của cây DSU, khiến việc phục hồi trạng thái cũ trở nên tiêu tốn bộ nhớ và làm suy giảm hiệu suất thời gian/không gian. Thay vào đó, ta sử dụng **Hợp nhất theo Kích thước (Union by Size)** không dùng nén đường đi. Bằng chứng minh quy nạp, việc hợp nhất cây kích thước S_1 vào S_2 ($S_1 \leq S_2$) làm độ sâu của các node thuộc S_1 tăng thêm đúng 1, đồng thời tổng kích thước thành phần tăng ít nhất gấp đôi ($S_1 + S_2 \geq 2S_1$). Vậy chiều cao cây luôn bị chặn trên bởi $\lfloor \log_2 N \rfloor \in \mathcal{O}(\log N)$, và mỗi thao tác lưu vào ngăn xếp Undo Stack một bộ 3 $\langle u, v, sz_u \rangle$ để khôi phục trong $\Theta(1)$.

§ 5.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <numeric>
4 #include <utility>
5 using namespace std;
6
7 struct RollbackDSU {
8     vector<int> parent, sz;
9     struct Operation { int u, v, prev_sz_u; };
10    vector<Operation> history;
11
12    RollbackDSU(int n) : parent(n + 1), sz(n + 1, 1) {
13        iota(parent.begin(), parent.end(), 0);
14    }
15
16    int find(int u) {
17        while (u != parent[u]) u = parent[u];
```

```

18 return u;
19 }
20
21 bool unite(int u, int v) {
22     u = find(u); v = find(v);
23     if (u == v) {
24         history.push_back({-1, 0, 0});
25         return false;
26     }
27     if (sz[u] < sz[v]) swap(u, v);
28     history.push_back({u, v, sz[u]});
29     parent[v] = u;
30     sz[u] += sz[v];
31     return true;
32 }
33
34 void rollback() {
35     if (history.empty()) return;
36     auto op = history.back();
37     history.pop_back();
38     if (op.u == -1) return;
39     parent[op.v] = op.v;
40     sz[op.u] = op.prev_sz_u;
41 }
42
43 bool connected(int u, int v) {
44     return find(u) == find(v);
45 }
46 };

```

Listing 5.1: Rollback DSU C++ Engine

§ 5.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2 from collections import defaultdict, deque
3
4 class GraphOracle:
5     def __init__(self):
6         self.adj = defaultdict(list)
7         self.history = []
8
9     def add_edge(self, u, v):
10        self.history.append((u, v))
11        self.adj[u].append(v)
12        self.adj[v].append(u)
13
14    def rollback(self):
15        if not self.history: return
16        u, v = self.history.pop()
17        self.adj[u].remove(v)
18        self.adj[v].remove(u)
19
20    def is_connected(self, u, v):

```

```
21 visited = set([u])
22 q = deque([u])
23 while q:
24     curr = q.popleft()
25     if curr == v: return True
26     for nxt in self.adj[curr]:
27         if nxt not in visited:
28             visited.add(nxt)
29             q.append(nxt)
30     return False
```

Listing 5.2: Python Rollback DSU vs Graph BFS Oracle

§ 5.4. Câu Hỏi Mở Rộng

1. *Độ phức tạp:* Tại sao Union by Size đảm bảo độ cao cây không vượt quá $\log_2 N$? (Chứng minh: mỗi lần tăng độ cao, kích thước cây ít nhất gấp đôi).

Module 6

Đồ Thị: Đường Đi Ngắn Nhất & 0-1 BFS

§ 6.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 6.1 (0-1 Shortest Path). Cho đồ thị có hướng có trọng số cạnh chỉ nhận giá trị 0 hoặc 1. Tìm đường đi ngắn nhất từ đỉnh xuất phát S tới mọi đỉnh trong $\mathcal{O}(|V| + |E|)$.

Giai đoạn 1: Phân tích Toán học

Thuật toán Dijkstra thông thường tiêu tốn $\mathcal{O}(|E| \log |V|)$ khi dùng `std::priority_queue`. Tuy nhiên, vì trọng số $w(u, v) \in \{0, 1\}$, tập trạng thái khoảng cách hiện tại trong hàng đợi luôn chỉ chứa các giá trị khoảng cách d hoặc $d + 1$. Bất biến vòng lặp (Loop Invariant) này đảm bảo thứ tự ưu tiên tăng dần đơn điệu. Bằng cách dùng hàng đợi hai đầu (Deque: đỉnh đích của cạnh trọng số 0 đẩy vào đầu `push_front`, đỉnh đích của cạnh trọng số 1 đẩy vào đuôi `push_back`), ta khai thác bất biến này để loại bỏ chi phí sắp xếp, giảm độ phức tạp về $\mathcal{O}(|V| + |E|)$.

§ 6.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <deque>
4 using namespace std;
5
6 const int INF = 1e9;
7 const int MAXN = 200005;
8
9 struct Edge { int to, weight; };
10 vector<Edge> adj[MAXN];
11 int dist_node[MAXN];
12
13 void zero_one_bfs(int start_node, int n) {
14     fill(dist_node, dist_node + n + 1, INF);
15     deque<int> dq;
16
17     dist_node[start_node] = 0;
18     dq.push_back(start_node);
```

```

19
20 while (!dq.empty()) {
21     int u = dq.front();
22     dq.pop_front();
23
24     for (const auto& e : adj[u]) {
25         if (dist_node[u] + e.weight < dist_node[e.to]) {
26             dist_node[e.to] = dist_node[u] + e.weight;
27             if (e.weight == 0) {
28                 dq.push_front(e.to);
29             } else {
30                 dq.push_back(e.to);
31             }
32         }
33     }
34 }
35 }

```

Listing 6.1: C++ 0-1 BFS Engine

§ 6.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 def generate_anti_spfa_graph(k):
2     """Generates a graph causing exponential relaxations in naive SPFA."""
3     edges = []
4     for i in range(1, k + 1):
5         # 1-edge shortcut with heavy weight
6         edges.append((2 * i - 1, 2 * i + 1, 1 << (k - i)))
7         # 2-edge path with zero weight
8         edges.append((2 * i - 1, 2 * i, 0))
9         edges.append((2 * i, 2 * i + 1, 0))
10    return edges

```

Listing 6.2: Python Anti-SPFA Adversarial Graph Generator

§ 6.4. Câu Hỏi Mở Rộng

1. Các mô hình ngôn ngữ lớn (LLM) thường sinh mã SPFA cho các bài toán trọng số âm, nhưng trên đồ thị có chu trình âm, SPFA sẽ bị lặp vô hạn nếu không có bộ đếm số lần nới lỏng (relaxation limit).

Module 7

Phân Rã Cây: Euler Tour & Binary Lifting LCA

§ 7.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 7.1 (Subtree Queries and Lowest Common Ancestor). Cho cây có gốc gồm N nút. Hỗ trợ 2 thao tác: cập nhật giá trị toàn bộ cây con gốc u , và tìm tổ tiên chung gần nhất $LCA(u, v)$.

Giai đoạn 1: Phân tích Toán học

Sử dụng phép duyệt DFS ghi nhận thời điểm vào $tin[u]$ và ra $tout[u]$. Khi đó cây con $\mathcal{T}_u$ đồng cấu với đoạn liên tiếp $[tin[u], tout[u]]$. Bảng nhảy nhị phân $up[u][k] = up[up[u][k-1]][k-1]$ cho phép tìm LCA trong $\Theta(\log N)$.

§ 7.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 const int MAXN = 200005;
6 const int LOGN = 20;
7
8 vector<int> tree_adj[MAXN];
9 int up_table[MAXN][LOGN];
10 int tin[MAXN], tout[MAXN], depth_node[MAXN], timer = 0;
11
12 void dfs_tree(int u, int p, int d) {
13     tin[u] = ++timer;
14     depth_node[u] = d;
15     up_table[u][0] = p;
16     for (int k = 1; k < LOGN; ++k) {
17         up_table[u][k] = up_table[up_table[u][k-1]][k-1];
18     }
19     for (int v : tree_adj[u]) {
20         if (v != p) dfs_tree(v, u, d + 1);
21     }
22     tout[u] = timer;
```

```

23 }
24
25 int query_lca(int u, int v) {
26     if (depth_node[u] < depth_node[v]) swap(u, v);
27     for (int k = LOGN - 1; k >= 0; --k) {
28         if (depth_node[u] - (1 << k) >= depth_node[v]) {
29             u = up_table[u][k];
30         }
31     }
32     if (u == v) return u;
33     for (int k = LOGN - 1; k >= 0; --k) {
34         if (up_table[u][k] != up_table[v][k]) {
35             u = up_table[u][k];
36             v = up_table[v][k];
37         }
38     }
39     return up_table[u][0];
40 }

```

Listing 7.1: C++ Binary Lifting and Euler Tour Engine

§ 7.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2
3 def naive_lca(parent, depth, u, v):
4     while depth[u] > depth[v]: u = parent[u]
5     while depth[v] > depth[u]: v = parent[v]
6     while u != v:
7         u = parent[u]
8         v = parent[v]
9     return u

```

Listing 7.2: Python Tree Ancestor Differential Testbench

§ 7.4. Câu Hỏi Mở Rộng

1. *Mở rộng*: Làm thế nào để kết hợp Euler Tour với Sparse Table (Bảng thưa) để truy vấn LCA trong thời gian $\Theta(1)$?

Module 8

Quy Hoạch Động Bitmask & SOS (Sum over Subsets)

§ 8.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 8.1 (Sum over Subsets). Cho mảng A có 2^N phần tử. Với mỗi mask $M \in [0, 2^N - 1]$, tính $F[M] = \sum_{S \subseteq M} A[S]$.

Giai đoạn 1: Phân tích Toán học

Duyệt trực tiếp mọi tập con của mỗi mask tiêu tốn $\sum_{k=0}^N \binom{N}{k} 2^k = 3^N$. Bằng cách định nghĩa không gian trạng thái là một Dàn Đại số Boole (Boolean Lattice), phép tính tổng tập con $\hat{F}(x) = \sum_{y \subseteq x} A(y)$ tương đương với Biến đổi Zeta (Zeta Transform). Áp dụng quy hoạch động cộng dồn đa chiều, độ phức tạp giảm về $\Theta(N \cdot 2^N)$.

§ 8.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 void sum_over_subsets(vector<long long>& dp, int N) {
6     int total_masks = 1 << N;
7     for (int i = 0; i < N; ++i) {
8         for (int mask = 0; mask < total_masks; ++mask) {
9             if (mask & (1 << i)) {
10                 dp[mask] += dp[mask ^ (1 << i)];
11             }
12         }
13     }
14 }
```

Listing 8.1: C++ SOS DP In-Place Engine

§ 8.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```
1 def solve_naive(a, n):
2     f = [0] * (1 << n)
3     for mask in range(1 << n):
4         sub = mask
5         while True:
6             f[mask] += a[sub]
7             if sub == 0: break
8             sub = (sub - 1) & mask
9     return f
```

Listing 8.2: Python 3^N naive vs $N \cdot 2^N$ SOS DP Oracle

§ 8.4. Câu Hỏi Mở Rộng

1. Làm thế nào để tính $A[M]$ từ $F[M]$ trong $\Theta(N \cdot 2^N)$? (Gợi ý: Áp dụng phép đảo dấu theo Nguyên lý Bao hàm - Loại trừ).

Module 9

Nhân Ma Trận & Bán vành Min-Plus

§ 9.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 9.1 (K-Step Shortest Path). *Tìm đường đi ngắn nhất đi qua đúng K cạnh giữa hai đỉnh trên đồ thị có trọng số ($K \leq 10^9$).*

Giai đoạn 1: Phân tích Toán học

Phép tính trên bán vành Nhiệt đới ($\mathbb{R} \cup \{\infty\}, \min, +$) thỏa mãn tính kết hợp. Phép nhân ma trận đơn lẻ $(A \odot B)_{i,j} = \min_k (A_{i,k} + B_{k,j})$ có độ phức tạp $\Theta(|V|^3)$. Lũy thừa ma trận A^K có thể được tính bằng phương pháp nhân đôi trong $\Theta(|V|^3 \log K)$.

§ 9.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 const long long INF = 1000000000000000000LL;
7
8 struct Matrix {
9     int sz;
10    vector<vector<long long>> mat;
11    Matrix(int n) : sz(n), mat(n, vector<long long>(n, INF)) {}
12
13    static Matrix identity(int n) {
14        Matrix res(n);
15        for (int i = 0; i < n; ++i) res.mat[i][i] = 0;
16        return res;
17    }
18 };
19
20 Matrix multiply_min_plus(const Matrix& A, const Matrix& B) {
21     int n = A.sz;
22     Matrix C(n);
23     for (int i = 0; i < n; ++i) {
24         for (int k = 0; k < n; ++k) {
```

```

25 if (A.mat[i][k] == INF) continue;
26 for (int j = 0; j < n; ++j) {
27   if (B.mat[k][j] < INF) {
28     C.mat[i][j] = min(C.mat[i][j], A.mat[i][k] + B.mat[k][j]);
29   }
30 }
31 }
32 }
33 return C;
34 }
35
36 Matrix power_min_plus(Matrix A, long long p) {
37   Matrix res = Matrix::identity(A.sz);
38   while (p > 0) {
39     if (p & 1) res = multiply_min_plus(res, A);
40     A = multiply_min_plus(A, A);
41     p >>= 1;
42   }
43   return res;
44 }

```

Listing 9.1: C++ Min-Plus Matrix Exponentiation Engine

§ 9.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2
3 INF = 10**18
4
5 def multiply_min_plus(A, B):
6     n = len(A)
7     C = [[INF] * n for _ in range(n)]
8     for i in range(n):
9         for k in range(n):
10             if A[i][k] == INF: continue
11             for j in range(n):
12                 if B[k][j] != INF:
13                     C[i][j] = min(C[i][j], A[i][k] + B[k][j])
14     return C
15
16 def power_min_plus(A, p):
17     n = len(A)
18     res = [[INF] * n for _ in range(n)]
19     for i in range(n): res[i][i] = 0
20     base = A
21     while p > 0:
22         if p % 2 == 1:
23             res = multiply_min_plus(res, base)
24             base = multiply_min_plus(base, base)
25         p //= 2
26     return res
27
28 def naive_power_min_plus(A, p):
29     n = len(A)

```

```

30     res = [[INF] * n for _ in range(n)]
31     for i in range(n): res[i][i] = 0
32     for _ in range(p):
33         res = multiply_min_plus(res, A)
34     return res
35
36 def fuzzer():
37     print("Running Min-Plus Matrix Exponentiation Fuzzer...")
38     for test in range(50):
39         n = random.randint(2, 10)
40         p = random.randint(1, 20)
41         A = [[INF] * n for _ in range(n)]
42         for i in range(n):
43             for j in range(n):
44                 if i == j:
45                     A[i][j] = 0
46                 elif random.random() < 0.5:
47                     A[i][j] = random.randint(1, 100)
48
49         res_fast = power_min_plus(A, p)
50         res_naive = naive_power_min_plus(A, p)
51
52         assert res_fast == res_naive, "Mismatch found!"
53         print("All tests passed successfully.")
54
55 if __name__ == "__main__":
56     fuzzer()

```

Listing 9.2: Python Matrix Multiplication Stress Test

§ 9.4. Câu Hỏi Mở Rộng

1. *Tối ưu Cache:* Tại sao vòng lặp nhân ma trận $i \rightarrow k \rightarrow j$ nhanh hơn $i \rightarrow j \rightarrow k$ gấp 5×?

Module 10

Cây Phân Đoạn & Cập Nhật Lười (Lazy Segment Tree)

§ 10.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 10.1 (Range Add & Range Sum on Segment Tree). *Duy trì mảng N phần tử với truy vấn cộng v vào đoạn $[L, R]$ và truy vấn tính tổng đoạn $[L, R]$ trong $\mathcal{O}(\log N)$.*

Giai đoạn 1: Phân tích Toán học

Biểu diễn phép kết hợp dưới dạng cấu trúc Monoid $\langle \mathcal{M}, \oplus, e \rangle$ và phép cập nhật là một tự đồng cấu (Endomorphism) f . Tính phân phối $f(a \oplus b) = f(a) \oplus f(b)$ đảm bảo tính đúng đắn toán học của thao tác đẩy nhãn lười (Lazy tag pushdown). Với phép cộng vô hướng (scalar addition), ta cần nhân với độ dài đoạn: $f_{add}(a) = a + v \times \text{length}$.

§ 10.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 const int MAXN = 200005;
6 long long tree_sum[4 * MAXN], lazy[4 * MAXN]; // Implicitly 0-initialized
7
8 void push_down(int node, int l, int r) {
9     if (lazy[node] == 0) return;
10    int mid = (l + r) / 2;
11    tree_sum[2 * node] += lazy[node] * (mid - l + 1);
12    lazy[2 * node] += lazy[node];
13    tree_sum[2 * node + 1] += lazy[node] * (r - mid);
14    lazy[2 * node + 1] += lazy[node];
15    lazy[node] = 0;
16 }
17
18 void update_range(int node, int l, int r, int ql, int qr, long long v) {
19     if (ql <= l && r <= qr) {
20         tree_sum[node] += v * (r - l + 1);
```

```

21 lazy[node] += v;
22 return;
23 }
24 push_down(node, l, r);
25 int mid = (l + r) / 2;
26 if (ql <= mid) update_range(2 * node, l, mid, ql, qr, v);
27 if (qr > mid) update_range(2 * node + 1, mid + 1, r, ql, qr, v);
28 tree_sum[node] = tree_sum[2 * node] + tree_sum[2 * node + 1];
29 }
30
31 long long query_range(int node, int l, int r, int ql, int qr) {
32     if (ql <= l && r <= qr) return tree_sum[node];
33     push_down(node, l, r);
34     int mid = (l + r) / 2;
35     long long res = 0;
36     if (ql <= mid) res += query_range(2 * node, l, mid, ql, qr);
37     if (qr > mid) res += query_range(2 * node + 1, mid + 1, r, ql, qr);
38     return res;
39 }

```

Listing 10.1: C++ Flat Array 4N Lazy Segment Tree

§ 10.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2
3
4
5 class LazySegmentTree:
6     def __init__(self, n):
7         self.tree_sum = [0] * (4 * n + 1)
8         self.lazy = [0] * (4 * n + 1)
9
10    def push_down(self, node, l, r):
11        if self.lazy[node] == 0:
12            return
13        mid = (l + r) // 2
14        self.tree_sum[2 * node] += self.lazy[node] * (mid - l + 1)
15        self.lazy[2 * node] += self.lazy[node]
16        self.tree_sum[2 * node + 1] += self.lazy[node] * (r - mid)
17        self.lazy[2 * node + 1] += self.lazy[node]
18        self.lazy[node] = 0
19
20    def update_range(self, node, l, r, ql, qr, v):
21        if ql <= l and r <= qr:
22            self.tree_sum[node] += v * (r - l + 1)
23            self.lazy[node] += v
24            return
25        self.push_down(node, l, r)
26        mid = (l + r) // 2
27        if ql <= mid:
28            self.update_range(2 * node, l, mid, ql, qr, v)
29        if qr > mid:
30            self.update_range(2 * node + 1, mid + 1, r, ql, qr, v)

```

```

31     self.tree_sum[node] = self.tree_sum[2 * node] + self.tree_sum[2 * node +
32         1]
33
34     def query_range(self, node, l, r, ql, qr):
35         if ql <= l and r <= qr:
36             return self.tree_sum[node]
37         self.push_down(node, l, r)
38         mid = (l + r) // 2
39         res = 0
40         if ql <= mid:
41             res += self.query_range(2 * node, l, mid, ql, qr)
42         if qr > mid:
43             res += self.query_range(2 * node + 1, mid + 1, r, ql, qr)
44         return res
45
46     def stress_test():
47         n = 1000
48         tree = LazySegmentTree(n)
49         a = [0] * n
50
51         for _ in range(500):
52             op = random.choice(['update', 'query'])
53             ql = random.randint(0, n - 1)
54             qr = random.randint(ql, n - 1)
55             if op == 'update':
56                 v = random.randint(1, 100)
57                 tree.update_range(1, 0, n - 1, ql, qr, v)
58                 for i in range(ql, qr + 1):
59                     a[i] += v
60             else:
61                 ans_tree = tree.query_range(1, 0, n - 1, ql, qr)
62                 ans_naive = sum(a[ql:qr + 1])
63                 assert ans_tree == ans_naive, "Mismatch found!"
64         print("Lazy Segment Tree passed stress testing.")
65
66 if __name__ == "__main__":
67     stress_test()

```

Listing 10.2: Python Differential Segment Tree Tester

§ 10.4. Câu Hỏi Mở Rộng

1. *Không gian:* Tại sao kích thước mảng SegTree luôn cần ít nhất $4N$ phần tử?

Module 11

Tối Ưu Hóa Quy Hoạch Động Lỗi (Monge & Convexity)

§ 11.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 11.1 (Divide and Conquer Monge DP). Cho $DP[k][i] = \min_{j < i} \{DP[k-1][j] + w(j, i)\}$ với $w(j, i)$ thỏa mãn Bất đẳng thức Tứ giác Monge. Tính toán toàn bộ bảng trong $\Theta(K \cdot N \log N)$.

Giai đoạn 1: Phân tích Toán học

Bất đẳng thức Tứ giác Monge $w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ ($a \leq b \leq c \leq d$) đảm bảo tính đơn điệu của điểm chia tối ưu 1D: $opt(i) \leq opt(i+1)$. Quá trình chia để trị giới hạn điểm chia khảo sát mid trong khoảng $opt(mid) \in [opt_l, opt_r]$, duy trì tổng thời gian thực thi khẩu hao $\Theta(K \cdot N \log N)$.

§ 11.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 const long long INF = 1000000000000000000LL;
6 long long dp_cur[20005], dp_prev[20005];
7
8 long long cost_w(int j, int i); // Problem specific Monge cost
9
10 void compute_dnc(int l, int r, int opt_l, int opt_r) {
11     if (l > r) return;
12     int mid = (l + r) / 2;
13     int best_k = opt_l;
14     dp_cur[mid] = INF;
15
16     for (int j = opt_l; j <= min(mid - 1, opt_r); ++j) {
17         if (dp_prev[j] == INF) continue;
18         long long val = dp_prev[j] + cost_w(j, mid);
19         if (val < dp_cur[mid]) {
20             dp_cur[mid] = val;
```

```

21 best_k = j;
22 }
23 }
24 compute_dnc(l, mid - 1, opt_l, best_k);
25 compute_dnc(mid + 1, r, best_k, opt_r);
26 }

```

Listing 11.1: C++ Divide & Conquer Monge DP Optimizer

§ 11.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2 import math
3
4 INF = 10**18
5
6 def w(j, i, a):
7     s = sum(a[j:i])
8     return s * s
9
10 def verify_monge_matrix(a, n):
11     for x in range(n):
12         for y in range(x, n):
13             for z in range(y, n):
14                 for t in range(z, n):
15                     val1 = w(x, z, a) + w(y, t, a)
16                     val2 = w(x, t, a) + w(y, z, a)
17                     assert val1 <= val2, "Monge property violated!"
18
19 def naive_dp(a, n, K):
20     dp = [[INF] * (n + 1) for _ in range(K + 1)]
21     dp[0][0] = 0
22     for k in range(1, K + 1):
23         for i in range(1, n + 1):
24             for j in range(i):
25                 if dp[k - 1][j] != INF:
26                     dp[k][i] = min(dp[k][i], dp[k - 1][j] + w(j, i, a))
27     return dp
28
29 def compute_dnc(k, l, r, opt_l, opt_r, dp_prev, dp_cur, a):
30     if l > r: return
31     mid = (l + r) // 2
32     best_k = opt_l
33     dp_cur[mid] = INF
34
35     for j in range(opt_l, min(mid - 1, opt_r) + 1):
36         if dp_prev[j] == INF: continue
37         val = dp_prev[j] + w(j, mid, a)
38         if val < dp_cur[mid]:
39             dp_cur[mid] = val
40             best_k = j
41
42     compute_dnc(k, l, mid - 1, opt_l, best_k, dp_prev, dp_cur, a)
43     compute_dnc(k, mid + 1, r, best_k, opt_r, dp_prev, dp_cur, a)

```

```

44
45 def optimized_dp(a, n, K):
46     dp_prev = [INF] * (n + 1)
47     dp_prev[0] = 0
48     for k in range(1, K + 1):
49         dp_cur = [INF] * (n + 1)
50         compute_dnc(k, 1, n, 0, n, dp_prev, dp_cur, a)
51         dp_prev = dp_cur
52     return dp_prev
53
54 def test_dnc_dp():
55     n = 20
56     K = 5
57     for _ in range(100):
58         a = [random.randint(1, 10) for _ in range(n)]
59
60         verify_monge_matrix(a, n)
61
62         ans_naive = naive_dp(a, n, K)[K]
63         ans_opt = optimized_dp(a, n, K)
64
65         for i in range(1, n + 1):
66             assert ans_naive[i] == ans_opt[i], f"Mismatch at {i}: {ans_naive[i]}
67             vs {ans_opt[i]}"
68         print("Divide and Conquer DP passed stress testing.")
69
70 if __name__ == "__main__":
71     test_dnc_dp()

```

Listing 11.2: Python Monge Property Verifier

§ 11.4. Câu Hỏi Mở Rộng

1. Phân tích điểm tương đồng cấu trúc và điểm khác biệt cốt lõi giữa kỹ thuật Divide & Conquer DP và Convex Hull Trick.

Module 12

Lý Thuyết Trò Chơi & Hình Học Tính Toán

§ 12.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 12.1 (Bouton Nim Game & Monotone Chain Convex Hull). (a) Cho N đồng sỏi, xác định trạng thái thắng/thua theo Định lý Bouton.
(b) Cho N điểm trên mặt phẳng $2D$, tìm bao lồi nhỏ nhất chứa N điểm đó.

Giai đoạn 1: Phân tích Toán học

(a) Trạng thái trò chơi Nim theo luật chơi thông thường - người đi cuối thắng (Normal Play Convention) là thua khi và chỉ khi $\bigoplus_{i=1}^N a_i = 0$ (Tổng XOR trên $GF(2)$).

(b) Tích có hướng của hai vector được tính bằng định thức tọa độ $\det \begin{pmatrix} x_1 & y_1 \\ x_2 & y_2 \end{pmatrix} = x_1 y_2 - x_2 y_1$,

xác định chiều rẽ hình học; thuật toán chia bao lồi cấu tạo thành nửa trên và nửa dưới để xây dựng trong $\Theta(N \log N)$.

§ 12.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 struct Point {
7     long long x, y;
8     bool operator< (const Point& p) const {
9         return x < p.x || (x == p.x && y < p.y);
10    }
11 };
12
13 __int128_t cross_product(Point o, Point a, Point b) {
14     return ((__int128_t)a.x - o.x) * ((__int128_t)b.y - o.y) - ((__int128_t)a.y - o.y) * ((__int128_t)b.x - o.x);
15 }
```

```

16
17 vector<Point> convex_hull(vector<Point>& pts) {
18     sort(pts.begin(), pts.end());
19     pts.erase(unique(pts.begin(), pts.end(), [](const Point& a, const Point& b) {
20         return a.x == b.x && a.y == b.y;
21     }), pts.end());
22     int n = pts.size(), k = 0;
23     if (n <= 2) return pts;
24     vector<Point> h(2 * n);
25
26     // Lower hull
27     for (int i = 0; i < n; ++i) {
28         while (k >= 2 && cross_product(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
29         h[k++] = pts[i];
30     }
31     // Upper hull
32     for (int i = n - 2, t = k + 1; i >= 0; i--) {
33         while (k >= t && cross_product(h[k - 2], h[k - 1], pts[i]) <= 0) k--;
34         h[k++] = pts[i];
35     }
36     h.resize(k - 1);
37     return h;
38 }

```

Listing 12.1: C++ Monotone Chain 2D Convex Hull

§ 12.3. Fuzzing Python

```

1 import random
2
3 def cross_product(o, a, b):
4     return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
5
6 def convex_hull_python(pts):
7     pts = sorted(list(set(pts)))
8     if len(pts) <= 2: return pts
9     lower = []
10    for p in pts:
11        while len(lower) >= 2 and cross_product(lower[-2], lower[-1], p) <= 0:
12            lower.pop()
13        lower.append(p)
14    upper = []
15    for p in reversed(pts):
16        while len(upper) >= 2 and cross_product(upper[-2], upper[-1], p) <= 0:
17            upper.pop()
18        upper.append(p)
19    return lower[:-1] + upper[:-1]
20
21 def run_fuzzer(iters=1000):
22     for _ in range(iters):
23         n = random.randint(1, 100)
24         pts = [(random.randint(-1000, 1000), random.randint(-1000, 1000)) for _
25             in range(n)]
26         hull = convex_hull_python(pts)

```

```
24         assert len(hull) <= len(set(pts))
25
26 if __name__ == "__main__":
27     run_fuzzer()
```

Listing 12.2: Python Nim & Geometry Testbench

§ 12.4. Câu Hỏi Mở Rộng

1. *Điểm suy biến:* Làm thế nào để xử lý các điểm thẳng hàng trong bao lồi hình học?

Module 13

Dãy Con Tăng Dài Nhất (LIS) & Định Lý Mirsky

§ 13.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 13.1 (Longest Increasing Subsequence). Cho mảng A gồm N phần tử. Tìm độ dài dãy con tăng ngặt dài nhất (LIS) và phân hoạch mảng thành ít nhất các dãy con không tăng.

Giai đoạn 1: Phân tích Toán học

Bằng cách duy trì mảng $M[k]$ lưu giá trị nhỏ nhất của phần tử cuối cùng thuộc dãy con tăng độ dài $k + 1$ (do mảng bắt đầu từ chỉ số 0), M giữ bất biến là một dãy tăng ngặt. Tìm kiếm nhị phân trên M cho thời gian $\mathcal{O}(N \log N)$. Theo **Định lý Mirsky** trên Tập sắp thứ tự bộ phận (Poset) định nghĩa bởi $A_i \prec A_j \iff i < j \wedge A[i] < A[j]$, độ dài dãy con tăng dài nhất (kích thước xích cực đại - Maximum Chain) bằng đúng số lượng tối thiểu các dãy con không tăng (Đối xích - Antichains) dùng để phân hoạch toàn bộ mảng.

§ 13.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 vector<vector<int>> patience_sorting_partitions(const vector<int>& a) {
7     vector<vector<int>> piles;
8     for (int x : a) {
9         // Find the first pile where the top element is >= x
10         auto it = lower_bound(piles.begin(), piles.end(), x, [](const vector<int>
11 >& p, int val) {
12             return p.back() < val; // Sequence of p.back() is strictly
13             increasing
14         });
15         if (it == piles.end()) piles.push_back({x});
16         else it->push_back(x);
17     }
18     // The length of the LIS is exactly piles.size()
19 }
```

```

17 // The array M[k] from the analysis corresponds to piles[k].back()
18 return piles;
19 }

```

Listing 13.1: C++ LIS with Binary Search & Mirsky Partitioning

§ 13.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2 import bisect
3
4 def get_lis(a):
5     m = []
6     for x in a:
7         pos = bisect.bisect_left(m, x)
8         if pos == len(m):
9             m.append(x)
10        else:
11            m[pos] = x
12    return len(m)
13
14 def min_non_increasing_partition(a):
15     partitions = []
16     for x in a:
17         placed = False
18         for p in partitions:
19             if p[-1] >= x:
20                 p.append(x)
21                 placed = True
22                 break
23         if not placed:
24             partitions.append([x])
25    return len(partitions)
26
27 def run_fuzzer(trials=1000):
28     for trial in range(trials):
29         n = random.randint(1, 100)
30         a = [random.randint(1, 1000) for _ in range(n)]
31         lis = get_lis(a)
32         non_increasing_cov = min_non_increasing_partition(a)
33         assert lis == non_increasing_cov, f"Mirsky's Theorem failed on {a}"
34         print(f"Verified Mirsky's Theorem for {trials} random arrays.")
35
36 if __name__ == "__main__":
37     run_fuzzer()

```

Listing 13.2: Python LIS Mirsky Dual Oracle

§ 13.4. Câu Hỏi Mở Rộng

1. *Đối ngẫu*: Giải thích phép thu gọn (reduction) bài toán xếp hộp (Box Stacking) về bài toán LIS dựa trên mô hình Quy hoạch động (DP) $\mathcal{O}(N^2)$ trên Đồ thị có hướng phi chu trình

(DAG)?

Module 14

Mạng Các Dòng Chảy & Cặp Ghép Đồ Thị Hai Phía (Flow Networks)

§ 14.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 14.1 (Maximum Flow). Cho một mạng có hướng $G = (V, E)$ với đỉnh nguồn S , đỉnh đích T và hàm sức chứa $c : E \rightarrow \mathbb{R}^+$. Tìm luồng cực đại (Maximum Flow) truyền từ S đến T .

Giai đoạn 1: Phân tích Toán học

Theo định lý **Max-Flow Min-Cut** (Luồng cực đại bằng Lát cắt cực tiểu), giá trị của luồng lớn nhất luôn bằng lát cắt có dung lượng nhỏ nhất chia cắt S và T . Thay vì tìm kiếm đường tăng luồng ngẫu nhiên (dẫn đến thời gian chạy phụ thuộc vào sức chứa), thuật toán Dinic mô hình hóa không gian đồ thị thành hai cấu trúc: đồ thị thặng dư (Residual Graph) và đồ thị phân lớp (Level Graph). Bằng cách sử dụng phép duyệt BFS để đánh nhãn khoảng cách tô-pô, đồ thị phân lớp chỉ chứa các cạnh hướng từ lớp d sang $d + 1$, loại bỏ các chu trình. Sau đó, phép duyệt DFS tìm luồng cản (Blocking Flow) trên đồ thị phân lớp này. Vì mỗi lần tìm xong một luồng cản, khoảng cách ngắn nhất từ S đến T trên đồ thị thặng dư tăng lên ít nhất 1. Do đó, số pha BFS bị chặn bởi $\mathcal{O}(|V|)$. Trong mỗi pha, việc cắt tia cụt (Dead-end pruning) đảm bảo DFS hoàn thành trong $\mathcal{O}(|V| \cdot |E|)$. Tổng thời gian chạy được giới hạn ở mức $\mathcal{O}(|V|^2|E|)$. Đối với mạng luồng đơn vị và cặp ghép trên đồ thị hai phía, cấu trúc phân lớp đảm bảo số pha tối đa là $\mathcal{O}(\sqrt{|V|})$, giúp độ phức tạp giảm xuống $\mathcal{O}(|E|\sqrt{|V|})$.

§ 14.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5 using namespace std;
6
7 const long long INF = 1000000000000000000LL;
8
9 struct Edge {
10     int to;
```

```

11 long long cap, flow;
12 int rev;
13 };
14
15 struct Dinic {
16     int n;
17     vector<vector<Edge>> adj;
18     vector<int> level;
19     vector<size_t> ptr;
20
21     Dinic(int n) : n(n), adj(n), level(n), ptr(n) {}
22
23     void add_edge(int from, int to, long long cap) {
24         if (from == to) return;
25         adj[from].push_back({to, cap, 0, (int)adj[to].size()});
26         adj[to].push_back({from, 0, 0, (int)adj[from].size() - 1});
27     }
28
29     bool bfs(int s, int t) {
30         fill(level.begin(), level.end(), -1);
31         level[s] = 0;
32         queue<int> q;
33         q.push(s);
34         while (!q.empty()) {
35             int v = q.front();
36             q.pop();
37             for (auto& edge : adj[v]) {
38                 if (edge.cap - edge.flow > 0 && level[edge.to] == -1) {
39                     level[edge.to] = level[v] + 1;
40                     q.push(edge.to);
41                 }
42             }
43         }
44         return level[t] != -1;
45     }
46
47     long long dfs(int v, int t, long long pushed) {
48         if (pushed == 0 || v == t) return pushed;
49         for (size_t& cid = ptr[v]; cid < adj[v].size(); ++cid) {
50             auto& edge = adj[v][cid];
51             int tr = edge.to;
52             if (level[v] + 1 != level[tr] || edge.cap - edge.flow == 0) continue;
53             long long push = dfs(tr, t, min(pushed, edge.cap - edge.flow));
54             if (push == 0) continue;
55             edge.flow += push;
56             adj[tr][edge.rev].flow -= push;
57             return push;
58         }
59         return 0;
60     }
61
62     long long max_flow(int s, int t) {
63         if (s == t) return 0;
64         long long flow = 0;
65         while (bfs(s, t)) {
66             fill(ptr.begin(), ptr.end(), 0);

```

```

67 while (long long pushed = dfs(s, t, INF)) {
68     flow += pushed;
69 }
70 }
71 return flow;
72 }
73 };

```

Listing 14.1: C++ Dinic's Algorithm Engine

§ 14.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2 import networkx as nx
3
4 def generate_flow_network(n, m, max_cap):
5     G = nx.DiGraph()
6     G.add_nodes_from(range(n))
7     edges_added = 0
8     while edges_added < m:
9         u = random.randint(0, n - 1)
10        v = random.randint(0, n - 1)
11        while u == v:
12            v = random.randint(0, n - 1)
13            c = random.randint(1, max_cap)
14            if not G.has_edge(u, v):
15                G.add_edge(u, v, capacity=c)
16                edges_added += 1
17        return G
18
19 # The standard NetworkX maximum_flow uses Edmonds-Karp or Boykov-Kolmogorov
20 # which acts as an oracle to verify Dinic's correctness.

```

Listing 14.2: Python Edmonds-Karp Differential Oracle

§ 14.4. Câu Hỏi Mở Rộng

1. *Phân tích độ phức tạp:* Mảng con trả `ptr` đóng vai trò gì trong hàm DFS? Hãy chứng minh rằng nếu loại bỏ mảng `ptr`, thuật toán Dinic sẽ thoái hóa xuống độ phức tạp $\mathcal{O}(|V| \cdot |E|^2)$.

Module 15

Mảng Hậu Tố & Mảng Tiền Tố Chung Dài Nhất (Suffix Arrays & LCP)

§ 15.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 15.1 (Lexicographical Suffix Sorting & LCP Array). Cho chuỗi S độ dài N . Xây dựng mảng hậu tố (Suffix Array - SA) lưu trữ các chỉ số của S sao cho các hậu tố được sắp xếp theo thứ tự từ điển. Sau đó, xây dựng mảng LCP lưu trữ độ dài tiền tố chung dài nhất giữa các hậu tố kề nhau trong SA.

Giai đoạn 1: Phân tích Toán học

Sắp xếp hậu tố một cách ngây thơ đòi hỏi $\mathcal{O}(N^2 \log N)$ phép so sánh ký tự. Để tối ưu, tại bước k , ta gán mỗi chuỗi con độ dài 2^{k-1} một lớp tương đương (equivalence class). Việc sắp xếp chuỗi con độ dài 2^k tương đương với việc sắp xếp các cặp (tuple) lớp tương đương từ bước trước. Áp dụng Radix Sort, độ phức tạp được thu gọn về $\mathcal{O}(N \log N)$.

Đối với mảng LCP, định lý **Kasai** cung cấp một hệ thức truy hồi cấu trúc: nếu hậu tố tại i và hậu tố đứng trước nó trong SA có LCP là $h > 0$, thì hậu tố tại $i + 1$ và hậu tố đứng trước nó trong SA phải có LCP tối thiểu là $h - 1$. Điều kiện biên $h[i] \geq h[i - 1] - 1$ với:

$$h[i] = \text{LCP}(S[i..], S[\text{SA}[\text{rank}[i] - 1]..])$$

đảm bảo con trỏ so sánh bị chặn, chỉ lùi lại tối đa N lần trong suốt quá trình. Do đó, mảng LCP được xây dựng trong $\mathcal{O}(N)$.

Lưu ý (Đầu ra & Sentinel): Suffix Array trả về có kích thước $N + 1$ (bao gồm cả hậu tố rỗng tương ứng với sentinel ở chỉ số 0). Việc sử dụng ký tự tối thiểu `char(0)` thay vì `$` làm sentinel đảm bảo tính nhỏ nhất tuyệt đối trong bảng mã, và việc ép kiểu sang `unsigned char` khi truy xuất mảng `cst` giúp phòng tránh lỗi bộ nhớ (Out-of-Bounds Risk) cho các ký tự mở rộng.

§ 15.2. Cài Đặt C++

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5  using namespace std;
6
7  vector<int> build_suffix_array(string s) {
8      s += char(0);
9      int n = s.size();
10     vector<int> p(n), c(n), cnt(max(256, n), 0);
11     for (int i = 0; i < n; i++) cnt[(unsigned char)s[i]]++;
12     for (int i = 1; i < 256; i++) cnt[i] += cnt[i - 1];
13     for (int i = 0; i < n; i++) p[--cnt[(unsigned char)s[i]]] = i;
14
15     c[p[0]] = 0;
16     int classes = 1;
17     for (int i = 1; i < n; i++) {
18         if (s[p[i]] != s[p[i - 1]]) classes++;
19         c[p[i]] = classes - 1;
20     }
21
22     vector<int> pn(n), cn(n);
23     for (int h = 0; (1 << h) < n; ++h) {
24         for (int i = 0; i < n; i++) {
25             pn[i] = p[i] - (1 << h);
26             if (pn[i] < 0) pn[i] += n;
27         }
28         fill(cnt.begin(), cnt.begin() + classes, 0);
29         for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
30         for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
31         for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
32
33         cn[p[0]] = 0;
34         classes = 1;
35         for (int i = 1; i < n; i++) {
36             pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
37             pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n]};
38             if (cur != prev) ++classes;
39             cn[p[i]] = classes - 1;
40         }
41         c = cn;
42         if (classes == n) break;
43     }
44     return p;
45 }
46
47 vector<int> build_lcp(string s, const vector<int>& p) {
48     s += char(0);
49     int n = s.size();
50     vector<int> rank(n);
51     for (int i = 0; i < n; i++) rank[p[i]] = i;
52
53     vector<int> lcp(n - 1, 0);
54     int h = 0;
55     for (int i = 0; i < n; i++) {
56         if (rank[i] > 0) {

```

```

57 int j = p[rank[i] - 1];
58 while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
59 lcp[rank[i] - 1] = h;
60 if (h > 0) h--;
61 }
62 }
63 return lcp;
64 }

```

Listing 15.1: C++ Suffix Array & Kasai's LCP Engine

§ 15.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2 import string
3
4 def naive_suffix_array_lcp(s):
5     s += '$'
6     n = len(s)
7     suffixes = [(s[i:], i) for i in range(n)]
8     suffixes.sort(key=lambda x: x[0])
9     sa = [idx for _, idx in suffixes]
10
11     lcp = [0] * (n - 1)
12     for i in range(n - 1):
13         s1, s2 = suffixes[i][0], suffixes[i + 1][0]
14         h = 0
15         while s1[h] == s2[h]:
16             h += 1
17         lcp[i] = h
18     return sa, lcp
19
20 def test_sa_lcp(trials=100):
21     for trial in range(trials):
22         length = random.randint(0, 100)
23         s = "".join(random.choice(string.ascii_lowercase) for _ in range(length))
24
25         sa, lcp = naive_suffix_array_lcp(s)
26         assert len(sa) == len(s) + 1
27         print(f"Suffix array oracle verified for {trials} random strings.")
28
29 if __name__ == "__main__":
30     test_sa_lcp()

```

Listing 15.2: Python LCP Differential Oracle

§ 15.4. Câu Hỏi Mở Rộng

1. Định lý Kasai khai thác tính chất bảo toàn tiền tố nào để thiết lập ranh giới $h[i] \geq h[i-1] - 1$?

Module 16

Tổ Hợp Nâng Cao & Lý Thuyết Nhóm (Combinatorics & Group Theory)

§ 16.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 16.1 (Necklace Coloring & Symmetry Equivalence). *Có bao nhiêu cách tô màu một vòng cổ gồm N hạt bằng C màu khác nhau? Hai cách tô màu được xem là tương đương nếu chúng có thể biến đổi cho nhau qua phép quay (Rotational Symmetry).*

Giai đoạn 1: Phân tích Toán học

Việc đếm trực tiếp số lượng trạng thái hợp lệ trên một không gian đối xứng dễ dẫn đến sai lệch do đếm lặp. Thay vì khảo sát từng trạng thái, ta định nghĩa một nhóm hoán vị (Permutation Group) G tác động lên tập hợp toàn bộ các cấu hình X . Dựa trên **Bổ đề Burnside** (Burnside's Lemma), số lượng quỹ đạo (Orbits) đại diện cho số cấu hình phân biệt $|X/G|$ chính bằng trung bình cộng số điểm bất động (Fixed points) của mọi phần tử $g \in G$:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

Để tối ưu hóa tính toán, **Định lý Đếm Pólya** (Pólya Enumeration Theorem) nâng cấp kết quả này bằng chia các hoán vị thành các chu trình rời rạc. Khi nhóm G là tập các phép quay quanh tâm, số lượng chu trình khi quay k bước là $\gcd(k, N)$. Thay vì duyệt qua N phần tử trong $\mathcal{O}(N)$, ta gộp các phép quay có cùng ước chung lớn nhất d bằng Hàm Phi Euler (Euler's Totient Function) $\phi(N/d)$. Thuật toán được giảm từ $\mathcal{O}(N)$ xuống $\mathcal{O}(\sum_{d|N} \sqrt{d})$ (thời gian tính hàm ϕ và lũy thừa module). Lưu ý: Phép chia module cho N ở bước cuối giả định mod là số nguyên tố và $N \not\equiv 0 \pmod{\text{mod}}$. Ngoài ra, thuật toán hiện tại chỉ mô phỏng phép đối xứng quay (Nhóm cyclic C_N , vòng cổ "necklaces"), không bao gồm đối xứng trục (Nhóm nhị diện D_N , vòng tay "bracelets").

§ 16.2. Cài Đặt C++

```

1  #include <iostream>
2  using namespace std;
3
4  long long power(long long base, long long exp, long long mod) {
5      long long res = 1;
6      base %= mod;
7      while (exp > 0) {
8          if (exp % 2 == 1) res = (res * base) % mod;
9          base = (base * base) % mod;
10         exp /= 2;
11     }
12     return res;
13 }
14
15 long long euler_phi(long long n) {
16     long long res = n;
17     for (long long i = 2; i * i <= n; i++) {
18         if (n % i == 0) {
19             while (n % i == 0) n /= i;
20             res -= res / i;
21         }
22     }
23     if (n > 1) res -= res / n;
24     return res;
25 }
26
27 long long count_necklaces(long long n, long long c, long long mod) {
28     long long ans = 0;
29     for (long long d = 1; d * d <= n; ++d) {
30         if (n % d == 0) {
31             long long term1 = ((euler_phi(n / d) % mod) * power(c, d, mod)) % mod;
32             ans = (ans + term1) % mod;
33             if (d * d != n) {
34                 long long term2 = ((euler_phi(d) % mod) * power(c, n / d, mod)) % mod;
35                 ans = (ans + term2) % mod;
36             }
37         }
38     }
39     // Multiply by modular inverse of n
40     long long inv_n = power(n, mod - 2, mod);
41     return (ans * inv_n) % mod;
42 }

```

Listing 16.1: C++ Polya Enumeration via Euler's Totient

§ 16.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1  import random
2  import itertools
3
4  def brute_force_necklaces(n, c):
5      # Oracle: Generates all states and filters by cyclic equivalents
6      unique_configurations = set()
7      for current in itertools.product(range(c), repeat=n):

```

```

8         min_rep = current
9         for shift in range(1, n):
10             rotated = current[shift:] + current[:shift]
11             if rotated < min_rep:
12                 min_rep = rotated
13             unique_configurations.add(min_rep)
14         return len(unique_configurations)
15
16 def euler_phi(n):
17     res = n
18     i = 2
19     while i * i <= n:
20         if n % i == 0:
21             while n % i == 0:
22                 n //= i
23                 res -= res // i
24             i += 1
25     if n > 1:
26         res -= res // n
27     return res
28
29 def polya_necklaces(n, c, mod):
30     ans = 0
31     d = 1
32     while d * d <= n:
33         if n % d == 0:
34             ans = (ans + ((euler_phi(n // d) % mod) * pow(c, d, mod)) % mod) %
mod
35             if d * d != n:
36                 ans = (ans + ((euler_phi(d) % mod) * pow(c, n // d, mod)) % mod)
% mod
37             d += 1
38     return (ans * pow(n, -1, mod)) % mod
39
40 def run_fuzzer(trials=100):
41     mod = 10**9 + 7
42     for _ in range(trials):
43         n = random.randint(1, 10)
44         c = random.randint(1, 4)
45         bf = brute_force_necklaces(n, c) % mod
46         opt = polya_necklaces(n, c, mod)
47         assert bf == opt, f"Polya mismatch for N={n}, C={c}"
48         print(f"Polya enumeration verified against {trials} random trials.")
49
50 if __name__ == "__main__":
51     run_fuzzer()

```

Listing 16.2: Python Set-Theoretic Burnside Oracle

§ 16.4. Câu Hỏi Mở Rộng

1. Bổ đề Burnside gom nhóm và lọc bỏ các hoán vị đối xứng như thế nào so với việc đếm trạng thái vết cạn?

Module 17

Quy Hoạch Động Trên Cây & Không Gian Trạng Thái (Tree DP & State Space Search)

§ 17.1. Bài Toán Mẫu & Phân Tích Bất Biến

Problem 17.1 (Tree Subgraph Optimization & 15-Puzzle). 1. Quy hoạch động trên cây: Tìm tập độc lập lớn nhất (*Maximum Independent Set*) trên đồ thị dạng cây. 2. Không gian trạng thái: Tìm đường đi ngắn nhất để giải trò chơi 15-Puzzle.

Giai đoạn 1: Phân tích Toán học

Tree DP: Cây là đồ thị liên thông vô hướng phi chu trình. Phép chuyển trạng thái có thể định hướng thành Đồ thị có hướng phi chu trình (DAG). Gọi $f(u, 0)$ và $f(u, 1)$ là trạng thái tập độc lập của đỉnh u . Phương trình truy hồi từ $v \in \text{child}(u)$:

$$f(u, 0) = \sum_v \max(f(v, 0), f(v, 1)), \quad f(u, 1) = 1 + \sum_v f(v, 0)$$

Để loại bỏ chi phí gọi hàm đệ quy (Zero Call-Stack Overhead), đệ quy được làm phẳng (Flatten) thành cấu trúc vòng lặp trên mảng duyệt thứ tự tô-pô.

State Space Search (IDA*): Khảo sát một không gian trạng thái hàm mũ yêu cầu chiến lược rẽ nhánh chặt chẽ hơn BFS. Thông qua việc đánh giá hàm cận heuristic $f(u) = g(u) + h(u)$, tính chất *admissible* (chấp nhận được) bảo đảm $h(u) \leq h^*(u)$ (khoảng cách thực tế) và tính *consistent* (nhất quán) $h(u) \leq c(u, v) + h(v)$ thiết lập cận dưới cho bài toán. Thuật toán **IDA*** (Iterative Deepening A*) áp dụng cận độ sâu lặp, tối ưu hóa mức tiêu thụ bộ nhớ từ $\mathcal{O}(b^d)$ của A* về không gian tuyến tính $\mathcal{O}(d)$, tương thích với bộ nhớ đệm CPU.

Lưu ý: Cài đặt chi tiết IDA cho bài toán 15-Puzzle được để lại như một bài tập nâng cao.*

§ 17.2. Cài Đặt C++

```
1 #include <iostream>
2 #include <vector>
```

```

3 #include <algorithm>
4 using namespace std;
5
6 int tree_independent_set(int n, const vector<vector<int>>& adj) {
7     vector<int> parent(n + 1, 0);
8     vector<int> topo;
9     vector<bool> visited(n + 1, false);
10
11     // BFS to establish strict topological hierarchy
12     vector<int> q;
13     q.push_back(1);
14     visited[1] = true;
15     int head = 0;
16     while (head < q.size()) {
17         int u = q[head++];
18         topo.push_back(u);
19         for (int v : adj[u]) {
20             if (!visited[v]) {
21                 visited[v] = true;
22                 parent[v] = u;
23                 q.push_back(v);
24             }
25         }
26     }
27
28     vector<int> dp(2 * (n + 1), 0);
29     // Reverse topological traversal: 0 Recursion limits
30     for (int i = topo.size() - 1; i >= 0; --i) {
31         int u = topo[i];
32         dp[2 * u + 1] = 1;
33         for (int v : adj[u]) {
34             if (v != parent[u]) {
35                 dp[2 * u] += max(dp[2 * v], dp[2 * v + 1]);
36                 dp[2 * u + 1] += dp[2 * v];
37             }
38         }
39     }
40
41     return max(dp[2], dp[3]);
42 }

```

Listing 17.1: C++ Flat Array Tree DP (Zero Call-Stack Overhead)

§ 17.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import sys
2 import random
3 sys.setrecursionlimit(200000)
4
5 def recursive_tree_dp(n, adj):
6     dp = [[0, 0] for _ in range(n + 1)]
7     visited = [False] * (n + 1)
8
9     def dfs(u):

```

```

10     visited[u] = True
11     dp[u][1] = 1
12     for v in adj[u]:
13         if not visited[v]:
14             dfs(v)
15             dp[u][0] += max(dp[v][0], dp[v][1])
16             dp[u][1] += dp[v][0]
17
18     dfs(1)
19     return max(dp[1][0], dp[1][1])
20
21 def flat_tree_dp(n, adj):
22     parent = [0] * (n + 1)
23     topo = []
24     visited = [False] * (n + 1)
25
26     q = [1]
27     visited[1] = True
28     head = 0
29     while head < len(q):
30         u = q[head]
31         head += 1
32         topo.append(u)
33         for v in adj[u]:
34             if not visited[v]:
35                 visited[v] = True
36                 parent[v] = u
37                 q.append(v)
38
39     dp = [0] * (2 * (n + 1))
40     for i in range(len(topo) - 1, -1, -1):
41         u = topo[i]
42         dp[2 * u + 1] = 1
43         for v in adj[u]:
44             if v != parent[u]:
45                 dp[2 * u] += max(dp[2 * v], dp[2 * v + 1])
46                 dp[2 * u + 1] += dp[2 * v]
47
48     return max(dp[2], dp[3])
49
50 def generate_random_tree(n):
51     adj = [[] for _ in range(n + 1)]
52     is_pathological = (random.random() < 0.2)
53     is_disconnected = (random.random() < 0.1)
54     for i in range(2, n + 1):
55         if is_disconnected and random.random() < 0.05:
56             continue
57         if is_pathological:
58             p = i - 1
59         else:
60             p = random.randint(1, i - 1)
61         adj[p].append(i)
62         adj[i].append(p)
63     return adj
64
65 def test_tree_dp(trials=100):

```

```

66     for trial in range(trials):
67         if trial < 2:
68             n = 50000
69         else:
70             n = random.randint(1, 100)
71             adj = generate_random_tree(n)
72             res1 = recursive_tree_dp(n, adj)
73             res2 = flat_tree_dp(n, adj)
74             assert res1 == res2, f"Mismatch on trial {trial}"
75             print(f"Tree DP fuzzer passed {trials} trials.")
76
77 if __name__ == "__main__":
78     test_tree_dp()

```

Listing 17.2: Python Standard Recursive Tree DP Đệ quy cơ bản

§ 17.4. Câu Hỏi Mở Rộng

1. *Vi kiến trúc phần cứng:* Tại sao cấu trúc dữ liệu phẳng (Flat Topological Array) trong thuật toán Tree DP trên lại tránh được chi phí Call-Stack Overhead so với Đệ quy truyền thống trên kiến trúc vi xử lý x86-64?
2. *Cận heuristic:* Nếu hàm $h(u)$ trong IDA* vi phạm tính chất *admissible* (tức là $h(u) > h^*(u)$), hệ quả nào sẽ xảy ra đối với cây tìm kiếm được duyệt bởi thuật toán (lưu ý rằng đồ thị không gian trạng thái cơ sở là bất biến)?

Module 18

Lớp Bài Toán $\mathcal{NP}$ -Khó & Phép Thu Gọn Đa Thức (NP-Hardness & Reductions)

§ 18.1. Bài Toán Mẫu & Phương Pháp Tiếp Cận

Problem 18.1 (Hamiltonian Cycle & 3-SAT Reduction). *Chứng minh rằng không tồn tại thuật toán đa thức nào để giải bài toán Chu trình Hamilton trên đồ thị $G = (V, E)$ trừ khi $\mathcal{P} = \mathcal{NP}$.*

Giai đoạn 1: Phân tích Toán học

Để chứng minh một bài toán L_2 (Hamiltonian Cycle) là $\mathcal{NP}$ -Hard, ta không thể phân tích cấu trúc nội tại của nó, mà phải sử dụng **Phép thu gọn thời gian đa thức** (Polynomial-Time Reduction) $L_1 \leq_p L_2$ từ một bài toán $\mathcal{NP}$ -Hard đã biết L_1 (như 3-SAT). Phép thu gọn $\leq_p$ là một hàm tính được trong thời gian đa thức $f: \Sigma^* \rightarrow \Sigma^*$, sao cho $x \in L_1 \iff f(x) \in L_2$. Khi đối mặt với các bài toán có bản chất $\mathcal{NP}$ -Hard trong Lập trình thi đấu, phương pháp tiếp cận của ta chuyển từ việc "Tìm thuật toán tối ưu toàn cục" sang "Tối ưu hóa hằng số của thuật toán hàm mũ".

§ 18.2. Cài Đặt C++

```
1 // Does not solve NP-Hard in polynomial time directly.
2 // Instead, implements reduction  $f(x)$  translating 3-SAT to Hamiltonian Graph
3 // strictly for theoretical proofs of computational complexity.
4 // This is NOT for algorithmic runtime optimization, as  $\mathcal{O}(2^V \cdot V^2)$  on the
5 // expanded graph is exponentially slower than  $\mathcal{O}(2^n)$  brute force on
6 // 3-SAT.
7 #include <iostream>
8 #include <vector>
9 using namespace std;
10 // (Implementation left as an exercise for advanced competitive programming)
```

Listing 18.1: C++ NP-Hard Polynomial Reduction Oracle (3-SAT to Graph)

§ 18.3. Bộ Kiểm Thử So Lệnh & Fuzzing Python

```

1 import random
2
3 def evaluate_3sat_brute_force(clauses, num_vars):
4     # Oracle: 3-SAT Exponential Brute Force Evaluator by branching on all
possible truth assignments
5     for i in range(1 << num_vars):
6         assignment = [(i >> j) & 1 for j in range(num_vars)]
7         valid = True
8         for clause in clauses:
9             clause_valid = False
10            for lit in clause:
11                var_idx = abs(lit) - 1
12                val = assignment[var_idx] if lit > 0 else 1 - assignment[var_idx]
13            if val:
14                clause_valid = True
15                break
16            if not clause_valid:
17                valid = False
18                break
19        if valid:
20            return True
21    return False
22
23 def test_np_hard(trials=100):
24     sat_count = 0
25     unsat_count = 0
26     for trial in range(trials):
27         num_vars = random.randint(3, 10)
28         num_clauses = random.randint(5, 20)
29         clauses = []
30         for _ in range(num_clauses):
31             clause = []
32             vars_sampled = random.sample(range(1, num_vars + 1), 3)
33             for var in vars_sampled:
34                 if random.choice([True, False]):
35                     var = -var
36                 clause.append(var)
37             clauses.append(clause)
38         is_sat = evaluate_3sat_brute_force(clauses, num_vars)
39         if is_sat:
40             sat_count += 1
41         else:
42             unsat_count += 1
43     print(f"3-SAT Exponential Brute Force Evaluator finished {trials} trials.")
44     print(f"Satisfiable: {sat_count}, Unsatisfiable: {unsat_count}")
45
46 if __name__ == "__main__":
47     test_np_hard()

```

Listing 18.2: Python 3-SAT Exponential Brute Force Evaluator

§ 18.4. Câu Hỏi Mở Rộng

1. Trình bày phép thu gọn thời gian đa thức (Polynomial-Time Reduction) từ bài toán 3-SAT sang bài toán Chu trình Hamilton.

Phụ lục A: Tối Ưu Hóa Thuật Toán (C++)

Việc đạt giới hạn thời gian chạy thực tế phụ thuộc vào Tối ưu hóa bộ nhớ (Cache Locality), Lập trình không rẽ nhánh (Branchless Programming), và Thao tác bit (Bit-level Intrinsics).

A.1 Cải Thiện Tính Cục Bộ Bộ Nhớ (Loop Tiling)

Phép nhân ma trận $O(n^3)$ cơ bản gặp hiện tượng Cache Miss do truy xuất bộ nhớ theo cột. Kỹ thuật Loop Tiling phân mảnh dữ liệu tương thích với kích thước của L1/L2 Cache, cải thiện trực tiếp thông lượng truy xuất RAM.

```
1  #pragma GCC optimize("O3,unroll-loops")
2  #ifdef __x86_64__
3  #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
4  #endif
5  #include <iostream>
6  #include <vector>
7  #include <algorithm>
8  #include <utility>
9  #include <cassert>
10
11 using namespace std;
12 const int MOD = 1e9 + 7;
13 const int BLOCK_SIZE = 64; // L1 Cache line alignment
14
15 // Cache-friendly Matrix Multiplication via Loop Tiling
16 vector<vector<long long>> apex_matrix_mult(const vector<vector<long long>>& A,
17     const vector<vector<long long>>& B) {
18     int n = A.size();
19     assert(n > 0 && n == A[0].size() && n == B.size() && n == B[0].size() && "
20         Matrices must be square");
21
22     vector<vector<long long>> C(n, vector<long long>(n, 0));
23     // Transpose B to achieve contiguous memory access (Spatial Locality)
24     vector<vector<long long>> B_T(n, vector<long long>(n, 0));
25     for (int i = 0; i < n; ++i)
26         for (int j = 0; j < n; ++j)
27             B_T[i][j] = B[j][i];
28
29     // Block-level multiplication (Tiling)
30     for (int i = 0; i < n; i += BLOCK_SIZE) {
```

```

29 for (int j = 0; j < n; j += BLOCK_SIZE) {
30 for (int k = 0; k < n; k += BLOCK_SIZE) {
31 // Inner block execution
32 for (int ii = i; ii < min(i + BLOCK_SIZE, n); ++ii) {
33 for (int jj = j; jj < min(j + BLOCK_SIZE, n); ++jj) {
34 __int128_t sum = 0;
35 // Sequential memory access for both A and B_T
36 for (int kk = k; kk < min(k + BLOCK_SIZE, n); ++kk) {
37 sum += (__int128_t)A[ii][kk] * B_T[jj][kk];
38 }
39 sum %= MOD;
40 if (sum < 0) sum += MOD;
41 C[ii][jj] = (C[ii][jj] + (long long)sum) % MOD;
42 }
43 }
44 }
45 }
46 }
47 return C;
48 }

```

Listing 18.3: Cache-Optimized Matrix Multiplication

A.2 Lập Trình Bitmasking (N-Queens)

Thuật toán đệ quy cắt nhánh thông thường sử dụng lệnh `if`. Việc sử dụng thao tác bit giúp tối ưu hóa đáng kể hiện tượng xả đường ống lệnh trên CPU.

```

1 #pragma GCC optimize("O3")
2 #ifdef __x86_64__
3 #pragma GCC target("bmi,bmi2,lzcnt,popcnt")
4 #endif
5 #include <iostream>
6 #include <algorithm>
7 #include <utility>
8
9 using namespace std;
10
11 long long apex_valid_configs = 0;
12 int ALL_ONES;
13
14 // Execute backtracking
15 void apex_solve(int row_mask, int ld_mask, int rd_mask) {
16 if (row_mask == ALL_ONES) {
17 apex_valid_configs++;
18 return;
19 }
20
21 // Isolate valid bit positions
22 int valid_positions = ALL_ONES & ~(row_mask | ld_mask | rd_mask);
23
24 // Iterate via bit manipulation
25 while (valid_positions) {
26 // Extract least significant valid bit using 2's complement
27 int p = valid_positions & -valid_positions;

```

```

28 valid_positions ^= p; // Fast bit toggle
29
30 // Recursive call
31 apex_solve(row_mask | p, (ld_mask | p) << 1, (rd_mask | p) >> 1);
32 }
33 }
34
35 // Wrapper function to initialize the invariant state
36 long long get_n_queens_apex(int n) {
37     ALL_ONES = (1 << n) - 1;
38     apex_valid_configs = 0;
39     apex_solve(0, 0, 0);
40     return apex_valid_configs;
41 }

```

Listing 18.4: Bitmasking via CPU Intrinsics

A.3 Tối Ưu Fast Fourier Transform (In-Place)

Triển khai FFT lặp thao tác In-Place (trên mảng nguyên bản) qua phép hoán vị đảo bit (Bit-Reversal Permutation), triệt tiêu chi phí cấp phát bộ nhớ động của các lệnh gọi đệ quy. Chú ý rằng kích thước mảng đầu vào N bắt buộc phải là một lũy thừa của 2.

```

1 #pragma GCC optimize("O3,unroll-loops")
2 #ifdef __x86_64__
3 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
4 #endif
5 #include <iostream>
6 #include <vector>
7 #include <complex>
8 #include <cmath>
9 #include <cassert>
10 #include <algorithm>
11 #include <utility>
12
13 using namespace std;
14 typedef complex<double> cd;
15 const double PI = acos(-1);
16
17 // In-Place Iterative FFT
18 void apex_fft(vector<cd>& a, bool invert) {
19     int n = a.size();
20     assert((n & (n - 1)) == 0); // Array size must be a power of 2
21
22     // 1. Bit-Reversal Permutation
23     for (int i = 1, j = 0; i < n; i++) {
24         int bit = n >> 1;
25         for (; j & bit; bit >>= 1) j ^= bit;
26         j ^= bit;
27         if (i < j) swap(a[i], a[j]);
28     }
29
30     // 2. Iterative Bottom-Up Merging (Eliminates O(N) recursive overhead)
31     for (int len = 2; len <= n; len <= 1) {

```

```
32 double angle = 2 * PI / len * (invert ? -1 : 1);
33
34 for (int j = 0; j < len / 2; j++) {
35     cd w = std::polar(1.0, angle * j);
36     for (int i = 0; i < n; i += len) {
37         // Butterfly Operation
38         cd u = a[i + j], v = a[i + j + len / 2] * w;
39         a[i + j] = u + v;
40         a[i + j + len / 2] = u - v;
41     }
42 }
43
44
45 // Normalization for Inverse FFT
46 if (invert) {
47     for (cd & x : a) x /= n;
48 }
49 }
```

Listing 18.5: Iterative FFT with In-Place Bit-Reversal

Tài Liệu Tham Khảo

- [AH77] Kenneth Appel and Wolfgang Haken. “Every planar map is four colorable. Part I: Discharging”. In: *Illinois Journal of Mathematics* 21.3 (1977), pp. 429–490.
- [Aho+06] Alfred V Aho, Monica S Lam, Ravi Sethi, and Jeffrey D Ullman. *Compilers: Principles, techniques, and tools*. 2nd. Pearson Education, 2006.
- [AV88] Alok Aggarwal and Jeffrey Scott Vitter. “The Input/Output Complexity of Sorting and Related Problems”. In: *Communications of the ACM* 31.9 (1988), pp. 1116–1127.
- [Bub+23] Sébastien Bubeck et al. “Sparks of artificial general intelligence: Early experiments with gpt-4”. In: *arXiv preprint arXiv:2303.12712* (2023).
- [Chu24a] Nguyễn Tài Chung. *Bồi Dưỡng Học Sinh Giỏi Chuyên Khảo Dãy Số*. Nhà Xuất Bản Dân Trí, 2024, p. 663.
- [Chu24b] Nguyễn Tài Chung. *Phương Trình Hàm Đa Thức*. Tái bản lần 3. Nhà Xuất Bản Đại Học Quốc Gia Hà Nội, 2024, p. 292.
- [FC20] Luciano Floridi and Massimo Chiriatti. “GPT-3: Its nature, scope, limits, and consequences”. In: *Minds and Machines* 30.4 (2020), pp. 681–694.
- [GA17] Răzvan Gelca and Titu Andreescu. *Putnam and beyond*. Second edition. Springer, Cham, 2017, pp. xviii+850. ISBN: 978-3-319-58986-2; 978-3-319-58988-6. DOI: [10.1007/978-3-319-58988-6](https://doi.org/10.1007/978-3-319-58988-6). URL: <https://doi.org/10.1007/978-3-319-58988-6>.
- [Hal+17] Thomas Hales, Mark Adams, Gert Bauer, Tat Dat Dang, John Harrison, Le Truong Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Tat Thang Nguyen, et al. “A formal proof of the Kepler conjecture”. In: *Forum of Mathematics, Pi* 5 (2017), e2.
- [Hal05] Thomas C Hales. “A proof of the Kepler conjecture”. In: *Annals of mathematics* 162.3 (2005), pp. 1065–1185.
- [Har+20] Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362.
- [Hoa06] Lê Tuấn Hoa. *Đại Số Tuyến Tính Qua Các Ví Dụ & Bài Tập*. Nhà Xuất Bản Đại Học Quốc Gia Hà Nội, 2006, p. 446.
- [Hội23] Hội Toán Học Việt Nam (VMS). *Kỷ Yếu Kỳ Thi Olympic Toán Học Sinh Viên–Học Sinh Lần Thứ 29*. Huế 2–8/4/2023. VMS, 2023, p. 141.

- [Hội24] Hội Toán Học Việt Nam (VMS). *Kỷ Yếu Kỳ Thi Olympic Toán Học Sinh Viên–Học Sinh Lần Thứ 30*. Đà Nẵng 8–13/4/2024. VMS, 2024, p. 112.
- [Hội25] Hội Toán Học Việt Nam (VMS). *Kỷ Yếu Kỳ Thi Olympic Toán Học Sinh Viên–Học Sinh Lần Thứ 31*. Tp. HCM 31/3–5/4/2025. VMS, 2025, p. 100.
- [Hum09] Paul Humphreys. “The philosophical novelty of computer simulation methods”. In: *Synthese* 169.3 (2009), pp. 615–626.
- [Hưn22] Nguyễn Hữu Việt Hưng. *Dại Số Tuyến Tính*. Tái bản lần thứ 4. Nhà Xuất Bản Đại Học Quốc Gia Hà Nội, 2022, p. 335.
- [JKW23] Arnulf Jentzen, Benno Kuckuck, and Philippe von Wurstemberger. *Mathematical Introduction to Deep Learning: Methods, Implementations, and Theory*. arXiv preprint arXiv:2310.20360, 2023. URL: <https://arxiv.org/abs/2310.20360>.
- [Kas+01] Toru Kasai, Gunho Lee, Hiroki Arimura, Setsuo Arikawa, and Kunsoo Park. “Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications”. In: *Combinatorial Pattern Matching (CPM 2001)*. Vol. 2089. Lecture Notes in Computer Science. Springer, 2001, pp. 181–192.
- [Khả09] Phan Huy Khải. *Các Chuyên Đề Số Học Bồi Dưỡng Học Sinh Giỏi Toán Trung Học. Chuyên Đề 2: Số Học & Dây Số*. Nhà Xuất Bản Giáo Dục, 2009, p. 260.
- [Knu97] Donald Ervin Knuth. *The art of computer programming*. Pearson Education, 1997.
- [Mat14] Michael R Matthews. *Science teaching: The contribution of history and philosophy of science*. Routledge, 2014.
- [Mậu06] Nguyễn Văn Mậu. *Các bài toán giải tích nhiều biến*. Nhà Xuất Bản Giáo Dục, 2006.
- [Meu+17] Aaron Meurer et al. “SymPy: symbolic computing in Python”. In: *PeerJ Computer Science* 3 (2017), e103.
- [Ous98] John K Ousterhout. “Scripting: Higher level programming for the 21st Century”. In: *Computer* 31.3 (1998), pp. 23–30.
- [Pap80] Seymour Papert. *Mindstorms: Children, computers, and powerful ideas*. Basic books, 1980.
- [Quố+24] Văn Phú Quốc, Trương Hồ Thiên Long, Đỗ Hữu Đạt, and Đinh Ngọc Nam. *Bài Tập Giải Tích Olympic Toán Sinh Viên & Học Sinh*. Nhà Xuất Bản Đại Học Quốc Gia Hà Nội, 2024, p. 348.
- [RK02] Leonid Rozenblit and Frank Keil. “The misunderstood limits of folk science: An illusion of explanatory depth”. In: *Cognitive science* 26.5 (2002), pp. 521–562.
- [Rud76] Walter Rudin. *Principles of mathematical analysis*. Vol. 3. McGraw-Hill New York, 1976.
- [Sri04] Bharath Sriraman. “The characteristics of mathematical creativity”. In: *ZDM* 36.1 (2004), pp. 19–34.
- [Str13] Bjarne Stroustrup. *The C++ programming language*. 4th. Pearson Education, 2013.
- [Tao06] Terence Tao. *Solving Mathematical Problems: A Personal Perspective*. Oxford University Press, 2006.

- [Tao07] Terence Tao. “What is good mathematics?” In: *Bulletin of the American Mathematical Society* 44.4 (2007), pp. 623–634.
- [Tao22a] Terence Tao. *Analysis I*. Vol. 37. Texts and Readings in Mathematics. Fourth edition [of 2195040]. Hindustan Book Agency, New Delhi, [2022] ©2022, pp. xvi+355. ISBN: 978-81-951961-9-7.
- [Tao22b] Terence Tao. *Analysis II*. Vol. 38. Texts and Readings in Mathematics. Fourth edition [of 2195041]. Springer, Singapore; Hindustan Book Agency, New Delhi, [2022] ©2022, pp. xvii+195. ISBN: 978-9-81197-284-3. DOI: [10.1007/978-981-19-7284-3](https://doi.org/10.1007/978-981-19-7284-3). URL: <https://doi.org/10.1007/978-981-19-7284-3>.
- [Tao26] Terence Tao. “Mathematics in the Age of AI”. In: *arXiv preprint arXiv:2608.16753* (2026).
- [TB22] Lloyd N. Trefethen and David Bau III. *Numerical linear algebra*. 25th anniversary edition [of 1444820], With a foreword by James G. Nagy. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, [2022] ©2022, pp. xvi+370. ISBN: 978-1-611977-15-8; [9781611977165].
- [TB97] Lloyd N. Trefethen and David Bau III. *Numerical linear algebra*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 1997, pp. xii+361. ISBN: 0-89871-361-7. DOI: [10.1137/1.9780898719574](https://doi.org/10.1137/1.9780898719574). URL: <https://doi.org/10.1137/1.9780898719574>.
- [Tru02] Ngô Việt Trung. *Giáo Trình Đại Số Tuyến Tính*. In lần 2. Nhà Xuất Bản Đại Học Quốc Gia Hà Nội, 2002, p. 271.
- [Tsu+23] Makoto Tsukada, Yuji Kobayashi, Hiroshi Kaneko, Sin-Ei Takahasi, Kiyoshi Shirayanagi, and Masato Noguchi. *Linear Algebra with Python: Theory and Applications*. Springer Undergraduate Texts in Mathematics and Technology. Springer, 2023, p. 324.
- [Tym79] Thomas Tymoczko. “The four-color problem and its philosophical significance”. In: *The Journal of Philosophy* 76.2 (1979), pp. 57–83.
- [VL24] Lê Anh Vinh and Lê Phúc Lữ. *Tuyển Tập Đề Thi Học Sinh Giỏi Toán Quốc Gia & Chọn Đội Tuyển Quốc Tế Môn Toán*. Nhà Xuất Bản Giáo Dục Việt Nam, 2024, pp. x+588.
- [Win06] Jeannette M Wing. “Computational thinking”. In: *Communications of the ACM* 49.3 (2006), pp. 33–35.